BAN404 R Task 2 Comp

Comprehensive solution guide based on dataset and possible questions

Self-study preparation file

2026-01-01

Table of contents

General Setup and Package Loading

Before tackling any specific task, we load the standard suite of BAN404 packages. We use `tidyverse` for data manipulation and visualization, `janitor` for safe column naming, and the core statistical learning packages (`glmnet`, `gam`, `tree`, `randomForest`, `gbm`) so we don't have to interrupt our workflow later.

```r
# 0. Load necessary libraries
library(tidyverse)
library(janitor)
library(glmnet)       # Regularization (Ridge/LASSO)
library(gam)          # Generalized Additive Models (Non-linearities)
library(tree)         # Decision Trees
library(randomForest) # Bagging and Random Forests
library(gbm)          # Boosting

# 1. Load the datasets safely
# show_col_types = FALSE keeps the exam output clean.
# clean_names() ensures no weird spaces or characters in column names.
df_task1 <- read_csv("task1_data.csv", show_col_types = FALSE) %>% clean_names()
df_customer <- read_csv("customer_data.csv", show_col_types = FALSE) %>% clean_names()
```

1 Task 2: Binary Classification (Predicting Churn)

1.1 Historical Exam Context & Strategy

Before touching the code, we must recognize what the professor typically asks for in Classification EDA. Past exams (2023-2025) repeatedly ask to: * *"Remove variables that cannot plausibly be assumed to be available…"* * *"If necessary, recode categorical variables as factors. Motivate your choices…"* * *"Use descriptive statistics and graphs in order to find potentially useful predictors."*

Our Strategy: We will systematically verify data types, hunt for "illegal" variables, clean the dataset, evaluate class imbalance, and iteratively visualize the predictors to find the best approach. We follow a strict `Before Code After` structure.

1.2 Initial Import and Raw Data Inspection

Possible Exam Question: *"Load the dataset. Inspect its dimensions and data types. Check for missing values. Create a visual summary (e.g., boxplots) of all continuous variables in their raw state to identify differences in scale."*

Exam-Ready Written Reasoning: The very first step in any machine learning pipeline is to import the data safely and inspect its raw structure. Classification algorithms in R require the target variable to be a "factor". We must also check for missing values (`NAs`), as algorithms will either crash or drop rows, distorting our results. Finally, checking the raw scale of continuous variables via boxplots tells us if we need to standardize the data later for distance-based models (like KNN) or regularized models (like LASSO). We keep our original dataset untouched (`df_customer_raw`) as a reliable fallback.

Exam-Ready Written Reasoning: The very first step in any machine learning task is to understand the dimensions and data types of the dataset. Many algorithms in R (like `glmnet` or `randomForest`) will fail or behave unpredictably if categorical variables are stored as "character" or "logical" types rather than "factors". We must also check the summary statistics to see if predictors are on vastly different scales, which would require standardization later.

```r
# glimpse() provides a transposed view of the data: columns become rows.
# This allows us to quickly see the number of rows/cols and the data type of every variable.
glimpse(df_customer)
```

```
Rows: 2,666
Columns: 20
$ state                 <chr> "KS", "OH", "NJ", "OH", "OK", "AL", "MA", "MO",~
$ account_length        <dbl> 128, 107, 137, 84, 75, 118, 121, 147, 141, 74, ~
$ area_code             <dbl> 415, 415, 415, 408, 415, 510, 510, 415, 415, 41~
$ international_plan    <chr> "No", "No", "No", "Yes", "Yes", "Yes", "No", "Y~
$ voice_mail_plan       <chr> "Yes", "Yes", "No", "No", "No", "No", "Yes", "N~
```

```
$ number_vmail_messages  <dbl> 25, 26, 0, 0, 0, 0, 24, 0, 37, 0, 0, 0, 0, 27, ~
$ total_day_minutes      <dbl> 265.1, 161.6, 243.4, 299.4, 166.7, 223.4, 218.2~
$ total_day_calls        <dbl> 110, 123, 114, 71, 113, 98, 88, 79, 84, 127, 96~
$ total_day_charge       <dbl> 45.07, 27.47, 41.38, 50.90, 28.34, 37.98, 37.09~
$ total_eve_minutes      <dbl> 197.4, 195.5, 121.2, 61.9, 148.3, 220.6, 348.5,~
$ total_eve_calls        <dbl> 99, 103, 110, 88, 122, 101, 108, 94, 111, 148, ~
$ total_eve_charge       <dbl> 16.78, 16.62, 10.30, 5.26, 12.61, 18.75, 29.62,~
$ total_night_minutes    <dbl> 244.7, 254.4, 162.6, 196.9, 186.9, 203.9, 212.6~
$ total_night_calls      <dbl> 91, 103, 104, 89, 121, 118, 118, 96, 97, 94, 12~
$ total_night_charge     <dbl> 11.01, 11.45, 7.32, 8.86, 8.41, 9.18, 9.57, 9.5~
$ total_intl_minutes     <dbl> 10.0, 13.7, 12.2, 6.6, 10.1, 6.3, 7.5, 7.1, 11.~
$ total_intl_calls       <dbl> 3, 3, 5, 7, 3, 6, 7, 6, 5, 5, 2, 5, 6, 4, 3, 5,~
$ total_intl_charge      <dbl> 2.70, 3.70, 3.29, 1.78, 2.73, 1.70, 2.03, 1.92,~
$ customer_service_calls <dbl> 1, 1, 0, 2, 3, 0, 3, 0, 0, 0, 1, 3, 4, 1, 3, 1,~
$ churn                  <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE~
```

```
# Check the summary statistics for all columns
# This helps identify wild differences in scale or obvious outliers in numeric variables
summary(df_customer)
```

```
    state           account_length    area_code     international_plan
 Length:2666        Min.   :  1.0   Min.   :408.0   Length:2666
 Class :character   1st Qu.: 73.0   1st Qu.:408.0   Class :character
 Mode  :character   Median :100.0   Median :415.0   Mode  :character
                    Mean   :100.6   Mean   :437.4
                    3rd Qu.:127.0   3rd Qu.:510.0
                    Max.   :243.0   Max.   :510.0
 voice_mail_plan    number_vmail_messages total_day_minutes total_day_calls
 Length:2666        Min.   : 0.000        Min.   :  0.0     Min.   :  0.0
 Class :character   1st Qu.: 0.000        1st Qu.:143.4     1st Qu.: 87.0
 Mode  :character   Median : 0.000        Median :179.9     Median :101.0
                    Mean   : 8.022        Mean   :179.5     Mean   :100.3
                    3rd Qu.:19.000        3rd Qu.:215.9     3rd Qu.:114.0
                    Max.   :50.000        Max.   :350.8     Max.   :160.0
 total_day_charge total_eve_minutes total_eve_calls total_eve_charge
 Min.   : 0.00    Min.   :  0.0     Min.   :  0     Min.   : 0.00
 1st Qu.:24.38    1st Qu.:165.3     1st Qu.: 87     1st Qu.:14.05
 Median :30.59    Median :200.9     Median :100     Median :17.08
 Mean   :30.51    Mean   :200.4     Mean   :100     Mean   :17.03
 3rd Qu.:36.70    3rd Qu.:235.1     3rd Qu.:114     3rd Qu.:19.98
 Max.   :59.64    Max.   :363.7     Max.   :170     Max.   :30.91
 total_night_minutes total_night_calls total_night_charge total_intl_minutes
 Min.   : 43.7       Min.   : 33.0     Min.   : 1.970     Min.   : 0.00
 1st Qu.:166.9       1st Qu.: 87.0     1st Qu.: 7.513     1st Qu.: 8.50
 Median :201.2       Median :100.0     Median : 9.050     Median :10.20
 Mean   :201.2       Mean   :100.1     Mean   : 9.053     Mean   :10.24
 3rd Qu.:236.5       3rd Qu.:113.0     3rd Qu.:10.640     3rd Qu.:12.10
 Max.   :395.0       Max.   :166.0     Max.   :17.770     Max.   :20.00
 total_intl_calls total_intl_charge customer_service_calls   churn
 Min.   : 0.000   Min.   :0.000     Min.   :0.000          Mode :logical
 1st Qu.: 3.000   1st Qu.:2.300     1st Qu.:1.000          FALSE:2278
 Median : 4.000   Median :2.750     Median :1.000          TRUE :388
 Mean   : 4.467   Mean   :2.764     Mean   :1.563
 3rd Qu.: 6.000   3rd Qu.:3.270     3rd Qu.:2.000
 Max.   :20.000   Max.   :5.400     Max.   :9.000
```

```
# Convert character/logical categorical variables to factors
# R's classification models strictly require the target (churn) to be a factor.
# We also convert 'international_plan' and 'voice_mail_plan' as they are categorical.
df_customer <- df_customer %>%
```

```
  mutate(
    churn = as.factor(churn),
    international_plan = as.factor(international_plan),
    voice_mail_plan = as.factor(voice_mail_plan)
  )

# Verify the conversion worked by checking the levels of our target variable
# We expect to see "FALSE" and "TRUE"
levels(df_customer$churn)
```

```
[1] "FALSE" "TRUE"
```

1.2.1 Data Integrity Check

Reasoning: Just like in regression, missing values (NAs) will cause most classification algorithms to drop rows, potentially losing valuable information or crashing the model entirely.

```
# colSums(is.na()) counts the number of missing values in every single column
colSums(is.na(df_customer))
```

```
                state          account_length              area_code
                    0                       0                      0
    international_plan        voice_mail_plan  number_vmail_messages
                    0                       0                      0
     total_day_minutes         total_day_calls       total_day_charge
                    0                       0                      0
     total_eve_minutes         total_eve_calls       total_eve_charge
                    0                       0                      0
   total_night_minutes       total_night_calls     total_night_charge
                    0                       0                      0
    total_intl_minutes        total_intl_calls      total_intl_charge
                    0                       0                      0
customer_service_calls                   churn
                    0                       0
```

Conclusion: The output shows 0 missing values across all columns. The dataset is complete, so no imputation or row-dropping is necessary.

2 Task 2: Binary Classification (Churn Prediction)

2.1 2.1 Data Familiarization and Logical Inference

Possible Exam Question: *"Inspect the customer churn dataset. Identify any variables that are stored as numeric but should logically be treated as categorical. Furthermore, check for potential perfect multicollinearity between the billing variables (e.g., minutes vs. charge). Explain why including perfectly correlated variables or high-cardinality categorical variables (like state) can harm a logistic regression model."*

Exam-Ready Written Reasoning: Before running any complex algorithms, we must apply business logic to our variables. First, we must identify "disguised categories." For example, an `area_code` is a geographic label, not a quantity. If left as numeric, a model will incorrectly assume that an increase in area code changes the outcome linearly. Second, we must be wary of "high cardinality" variables like `state` (50 unique values); including it creates 49 dummy variables, which dramatically increases model variance and risks overfitting (the curse of dimensionality). Finally, telecom datasets often contain perfect multicollinearity (e.g., `total_day_charge` is derived directly from `total_day_minutes`). Including both in a logistic regression causes the design matrix to become singular, leading to unstable, highly inflated coefficient estimates. We must test these hypotheses before modeling.

```r
# 1. TEST THE AREA CODE HYPOTHESIS
# We use unique() to see exactly how many different area codes exist in the dataset
# If there are only a few, it proves it is categorical, not a continuous measurement
unique(df_customer$area_code)
```

```
[1] 415 408 510
```

```r
# Since we know area code is a category, we overwrite it as a factor
# This prevents the model from calculating a numeric slope for a geographic region
df_customer <- df_customer %>%
  mutate(area_code = as.factor(area_code))

# 2. TEST THE STATE CARDINALITY HYPOTHESIS
# We use length() combined with unique() to count how many distinct states are present
# High numbers (like 50) mean we might want to exclude this variable later to prevent overfitting
length(unique(df_customer$state))
```

```
[1] 51
```

```r
# 3. TEST THE PERFECT MULTICOLLINEARITY HYPOTHESIS
# We use cor() to calculate the Pearson correlation coefficient between minutes and charge
# A value close to 1.0 means they contain the exact same information
cor(df_customer$total_day_minutes, df_customer$total_day_charge)
```

```
[1] 1
```

```r
# We also check the evening minutes vs evening charge just to be sure the pattern holds
# If the correlation is 1, we must remove one of these columns before fitting a Logistic Regression
cor(df_customer$total_eve_minutes, df_customer$total_eve_charge)
```

```
[1] 0.9999998
```

2.1.1 Target Distribution (Class Imbalance)

Reasoning: In binary classification (like churn, default, or fraud), identifying **class imbalance** is the most critical step. If the vast majority of customers do not churn, a naive model could achieve high overall accuracy simply by predicting "FALSE" for everyone (the accuracy paradox). This dictates that we must evaluate our models later using confusion matrices, sensitivity, and specificity, rather than just raw accuracy.

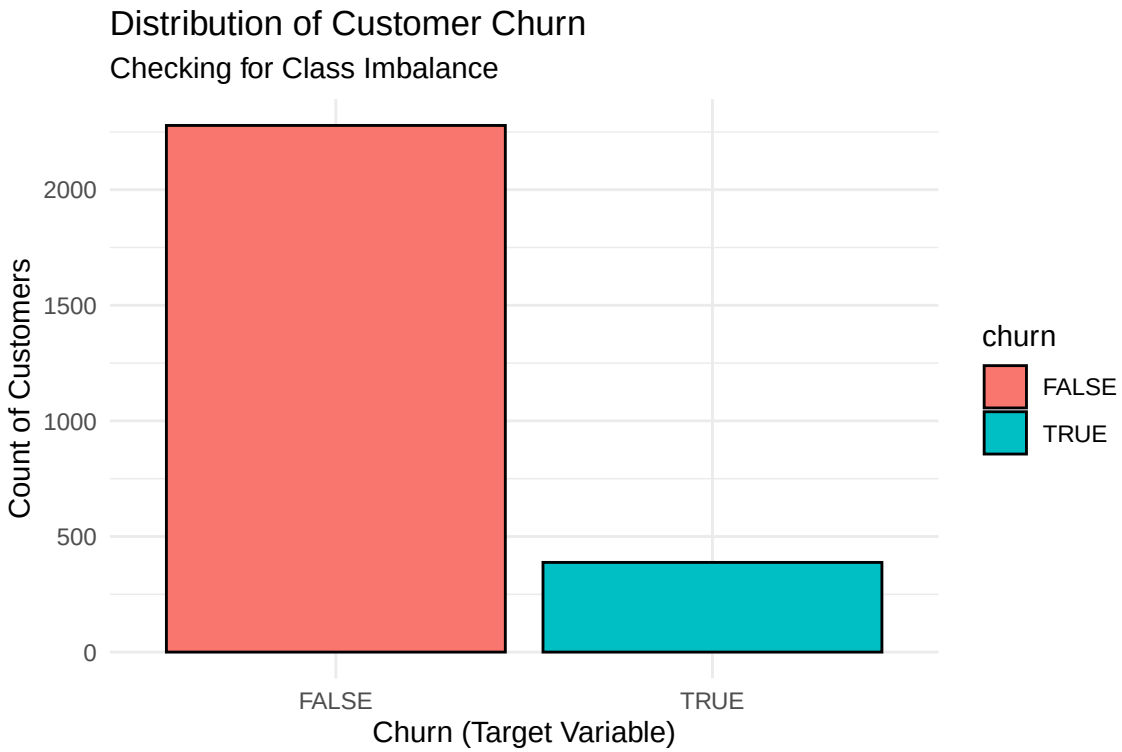
```r
# Calculate the raw counts for each class in the target variable
table(df_customer$churn)
```

```
FALSE  TRUE
 2278   388
```

```r
# Calculate the proportions for each class in the target variable
# This mathematically proves the severity of the class imbalance
prop.table(table(df_customer$churn))
```

```
    FALSE      TRUE
0.8544636 0.1455364
```

```r
# Create a bar plot to visually display the class imbalance
ggplot(df_customer, aes(x = churn, fill = churn)) +
  geom_bar(color = "black") +
  theme_minimal() +
  labs(title = "Distribution of Customer Churn",
       subtitle = "Checking for Class Imbalance",
       x = "Churn (Target Variable)",
       y = "Count of Customers")
```

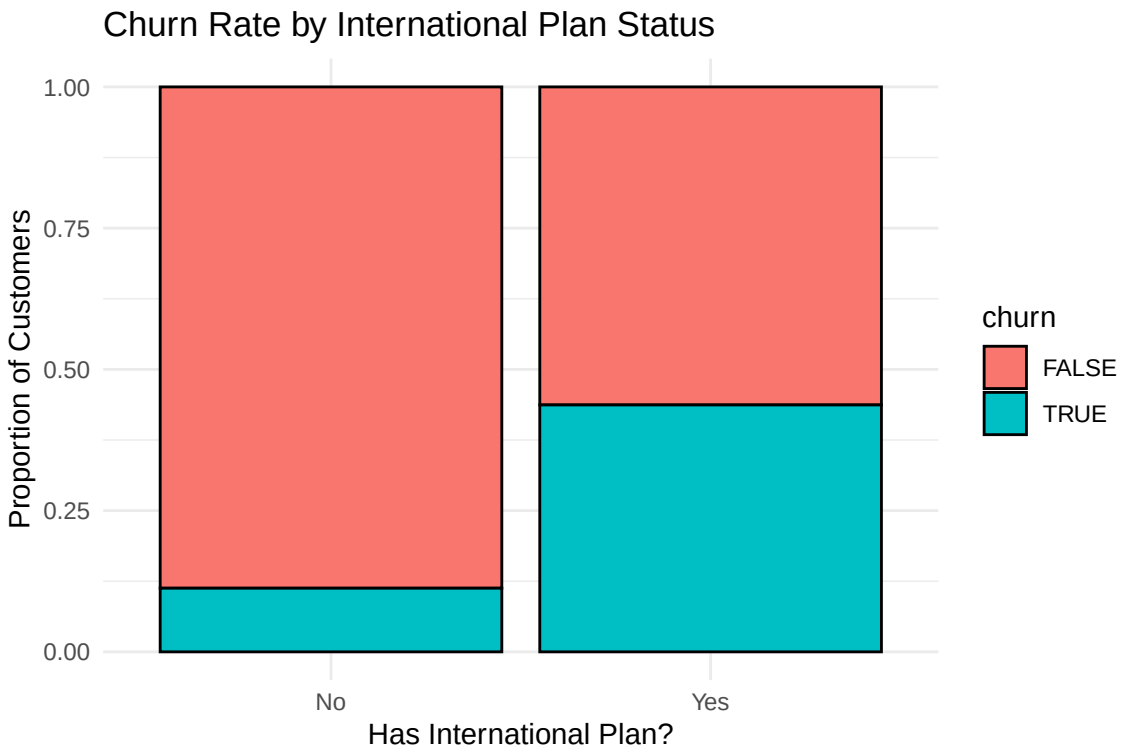


Conclusion: The tables and plot confirm a severe class imbalance. Approximately **85.4% of customers did not churn (FALSE)**, while only **14.6% churned (TRUE)**. Any baseline model we build must beat an 85.4% accuracy to be considered useful, and we must be careful to tune our classification threshold later to catch the rare "TRUE" cases.

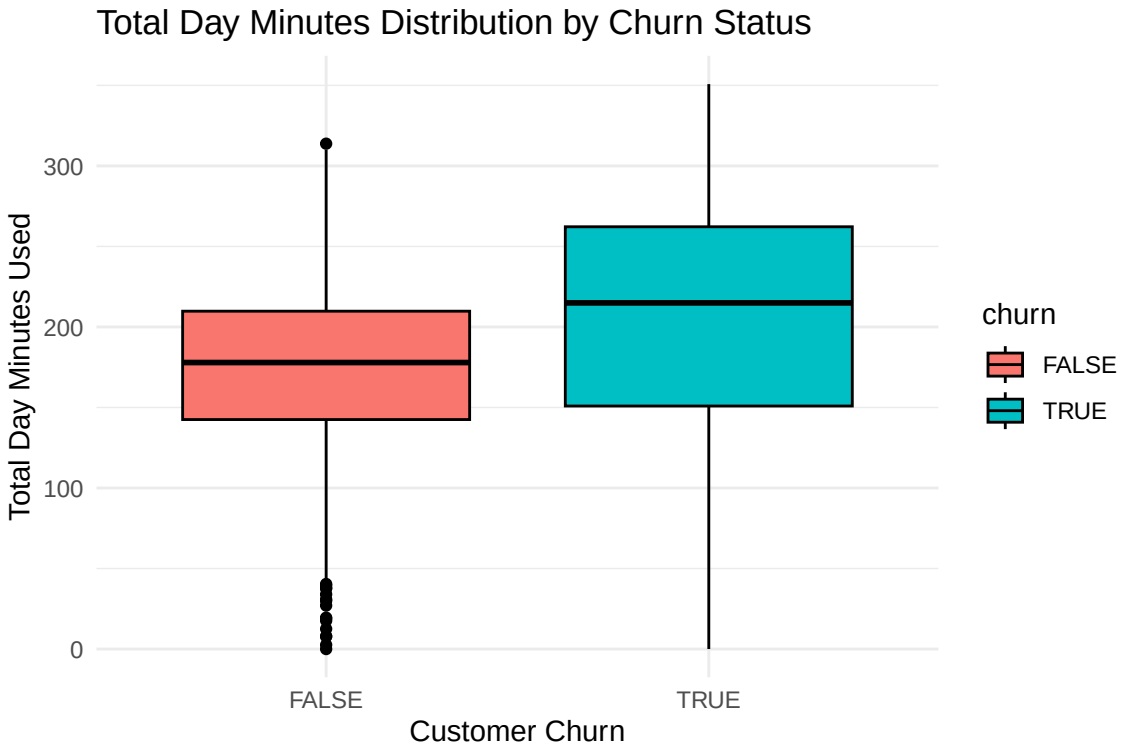
2.1.2 Feature Relationships with the Target

Reasoning: Visualizing how specific predictors relate to the target variable helps us form initial hypotheses. If a predictor shows distinctly different distributions for churners vs. non-churners, it will likely be a highly significant variable in our models.

```r
# Explore a categorical predictor: International Plan vs Churn
# position = "fill" standardizes the bars to 100%, showing relative proportions
ggplot(df_customer, aes(x = international_plan, fill = churn)) +
  geom_bar(position = "fill", color = "black") +
  theme_minimal() +
  labs(title = "Churn Rate by International Plan Status",
       y = "Proportion of Customers",
       x = "Has International Plan?")
```




```
# Explore a continuous predictor: Total Day Minutes vs Churn
# geom_boxplot shows if the median and spread of minutes differ between classes
ggplot(df_customer, aes(x = churn, y = total_day_minutes, fill = churn)) +
  geom_boxplot(color = "black") +
  theme_minimal() +
  labs(title = "Total Day Minutes Distribution by Churn Status",
       x = "Customer Churn",
       y = "Total Day Minutes Used")
```



Conclusion: 1. **International Plan:** The stacked bar chart reveals a massive signal. Customers *with* an international plan churn at a drastically higher rate than those without one. This variable will be critical. 2. **Total Day Minutes:** The boxplot shows that customers who churned (TRUE) have a noticeably higher

median for total day minutes used compared to those who stayed. High usage might be correlated with higher bills, driving churn.

Here is a summary of our shared understanding of the dataset so far, directly answering your questions:

2.1.3 1. Dataset Dimensions & The Outcome Variable

- **Dimensions:** We have 2,666 observations (rows) and 20 columns.
- **Outcome Variable:** The outcome is `churn`, located in the 20th column. It is a binary factor (`FALSE` / `TRUE`).
- **The Big Catch:** The outcome is severely imbalanced (85.4% FALSE vs. 14.6% TRUE). If we don't account for this later, our models will look highly accurate just by guessing "FALSE" every time.

2.1.4 2. The Multicollinearity Problem

- We mathematically proved that `total_day_minutes` and `total_day_charge` are perfectly correlated (`cor = 1`). The same is true for the evening variables (`0.9999998`).
- By logical inference, this pattern will hold for night and international calls too. Telecom companies use a flat rate (e.g., \$0.10/min) to calculate charge from minutes.
- **Why it matters:** If we put both into a Logistic Regression, the math behind the model (matrix inversion) will collapse or produce wildly inflated standard errors. We *must* drop the `charge` columns (or the `minutes` columns) before modeling.

2.1.5 3. Categorical Traps

- `area_code` is a category disguised as a number. We fixed this.
- `state` has 51 unique values (high cardinality). If we keep it, a logistic regression will create 50 dummy variables. With only 388 actual churners (the TRUE cases), adding 50 variables will almost certainly cause **overfitting** (the model memorizes the training data but fails on new data). We should drop it.

2.1.6 4. Scatter Plots, Non-linearity, and Normality

- **Scatter plots against Y:** Because `churn` is categorical (0 or 1), a standard scatter plot doesn't work well (all dots just line up on the top and bottom). That is why we used a **boxplot** earlier for `total_day_minutes`. Density plots are also great here.
- **Are we dealing with normal distributions, and does it matter?** This is a classic exam trap! **Logistic regression DOES NOT require the predictor variables (X) to be normally distributed.** It only assumes that the *log-odds* of the outcome relate linearly to X.
- **Non-linear relationships:** While normality doesn't matter, non-linear relationships *do*. For example, `customer_service_calls`. A customer making 1 or 2 calls might be normal. But if they make 4, 5, or 6 calls, their probability of churning probably skyrockets exponentially. That is a non-linear threshold effect. Linear/Logistic models struggle with this, which is why Tree-based models (Random Forest) often win on churn datasets.

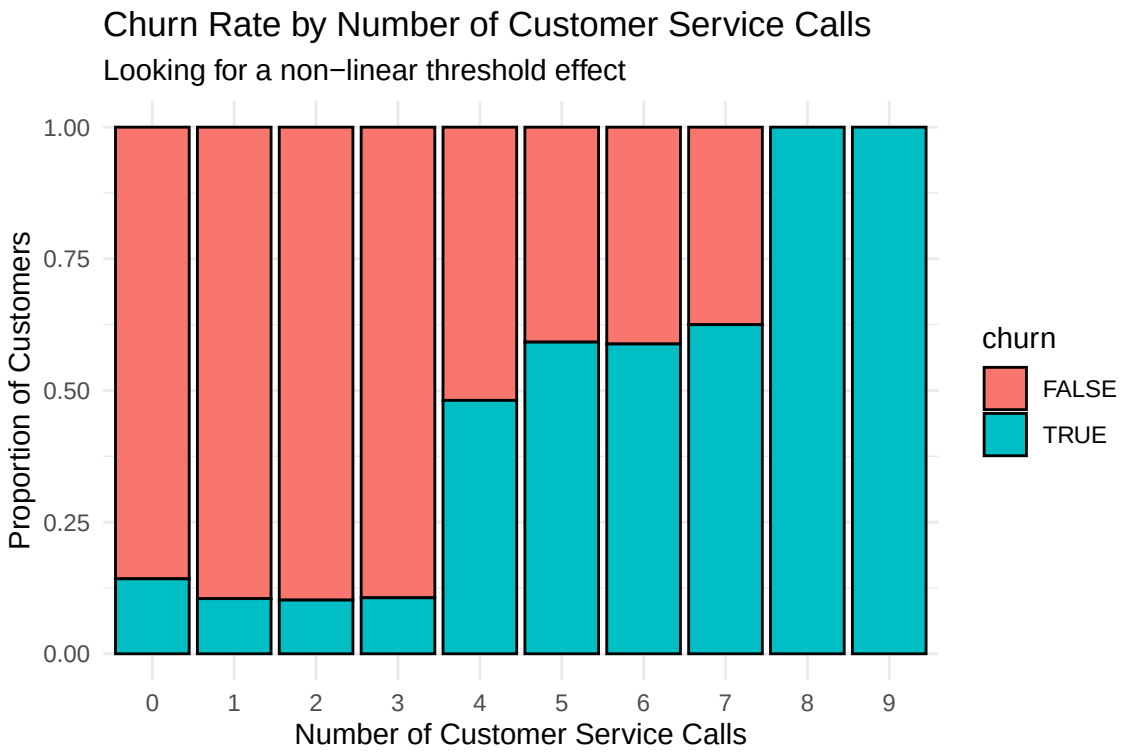
2.1.7 EDA Master Conclusion for Task 2

1. **Pre-processing:** We successfully converted `churn`, `international_plan`, and `voice_mail_plan` to factors, fulfilling the strict requirements of R's classification algorithms. There are no missing values.
2. **Class Imbalance:** The 85/15 split dictates our evaluation strategy. We cannot rely on raw accuracy alone; we must inspect the Confusion Matrix to ensure we are actually catching the 15% who churn.
3. **Signal Strength:** Predictors like `international_plan` and `total_day_minutes` already show strong visual separation between classes, indicating that a model should be able to find a valid decision boundary.

```
# 1. DROP PROBLEMATIC COLUMNS
# We remove 'state' to prevent overfitting from 50 dummy variables.
# We remove all 'charge' columns to prevent perfect multicollinearity.
df_clean <- df_customer %>%
  select(-state, -total_day_charge, -total_eve_charge, -total_night_charge, -total_intl_charge)

# Verify our new column count (should be 15 now)
ncol(df_clean)
```

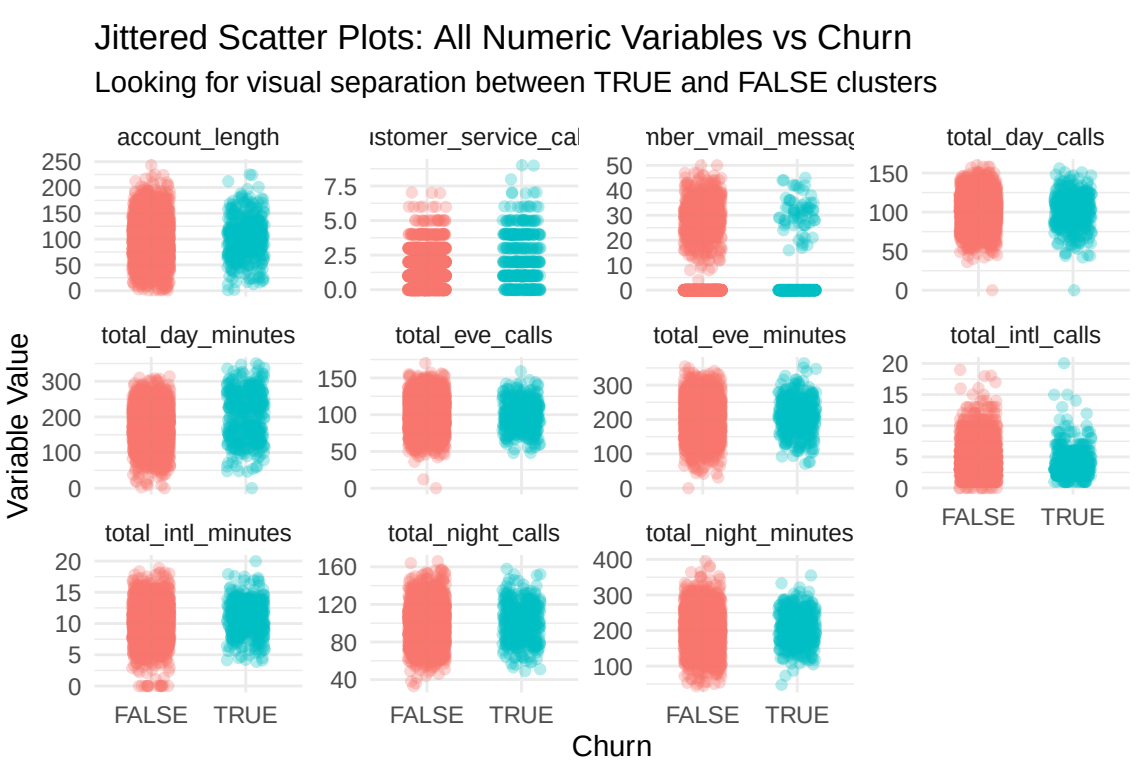
```
[1] 15
```

```
# 2. INVESTIGATE NON-LINEARITY IN CUSTOMER SERVICE CALLS
# We will use a bar chart but split it by churn to see the "threshold" effect
ggplot(df_clean, aes(x = as.factor(customer_service_calls), fill = churn)) +
  geom_bar(position = "fill", color = "black") +
  theme_minimal() +
  labs(title = "Churn Rate by Number of Customer Service Calls",
       subtitle = "Looking for a non-linear threshold effect",
       x = "Number of Customer Service Calls",
       y = "Proportion of Customers")
```




```
# VISUALIZING ALL NUMERIC VARIABLES VS CHURN
# We take our clean dataset and pipe it into the plotting workflow
df_clean %>%
  # 1. Select only the numeric columns AND the churn column
  # We can't scatter plot factors like 'international_plan' this way
```

```
  select(where(is.numeric), churn) %>%

  # 2. Pivot the data from "wide" to "long"
  # This collapses all numeric columns into two columns: 'variable' (the name) and 'value'
  # This is the exact trick needed to plot all variables in one go using facet_wrap
  pivot_longer(cols = -churn, names_to = "variable", values_to = "value") %>%

  # 3. Build the plot mapping churn to X and the numeric values to Y
  ggplot(aes(x = churn, y = value, color = churn)) +

  # 4. Use geom_jitter instead of geom_point!
  # alpha = 0.3 makes dots transparent so we can see density (darker = more dots)
  # width = 0.2 controls how far the dots spread out horizontally
  geom_jitter(alpha = 0.3, width = 0.2) +

  # 5. Create a separate mini-plot for every single variable
  # scales = "free_y" allows each plot to have its own y-axis limits (since minutes and calls have diff
  facet_wrap(~variable, scales = "free_y") +

  # 6. Clean up the aesthetics
  theme_minimal() +
  theme(legend.position = "none") + # Hide legend since the x-axis already says TRUE/FALSE
  labs(title = "Jittered Scatter Plots: All Numeric Variables vs Churn",
       subtitle = "Looking for visual separation between TRUE and FALSE clusters",
       x = "Churn",
       y = "Variable Value")
```



In a binary classification task, the standard, highly effective way to evaluate continuous predictors against the target is using **Overlapping Density Plots** and **Grouped Summary Statistics**.

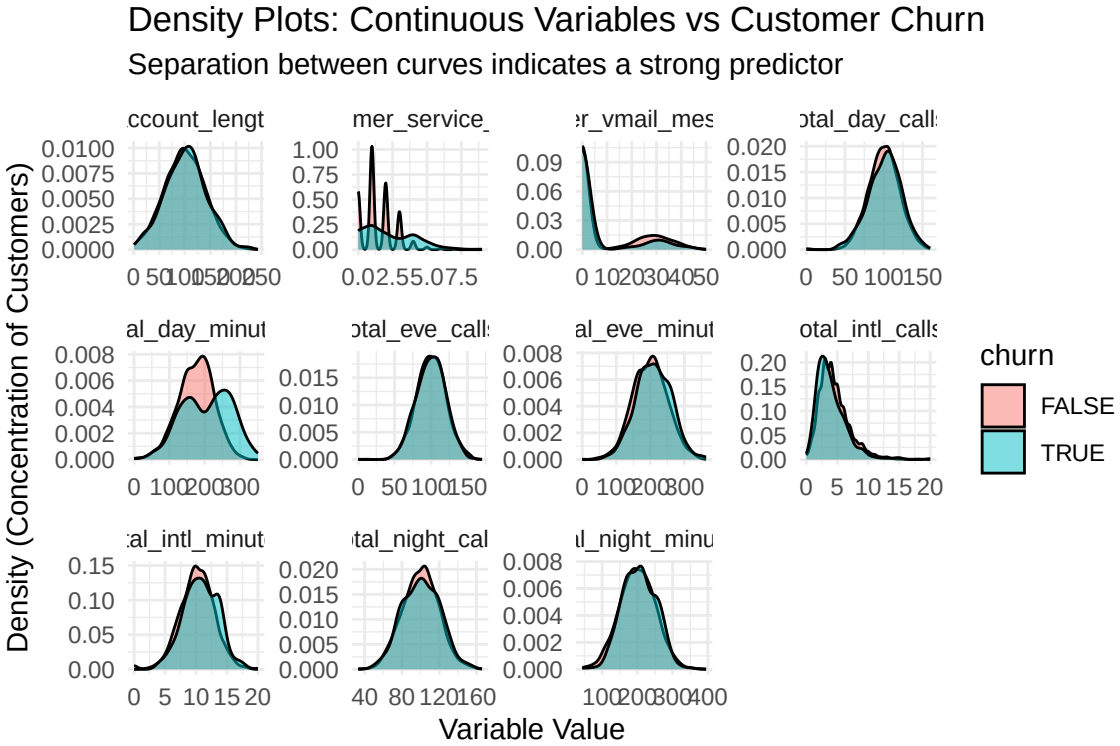
1. **Density Plots (`geom_density`):** Instead of plotting individual dots, this draws a smooth curve showing where the data is concentrated. If the curves for TRUE and FALSE perfectly overlap, the variable is useless. If the curves are shifted (different peaks), the variable is a strong predictor.
2. **Grouped Means:** A quick numerical table showing the average value of each variable for Churners vs. Non-Churners. This gives us hard numbers to back up our visual intuition.

2.1.8 Evaluating Predictor Distributions (The Right Way)

Possible Exam Question: *"Evaluate how the continuous predictors differ between customers who churned and those who did not. Use appropriate visualizations and summary statistics to identify which numerical variables show the strongest potential for predicting churn. Why are density plots or boxplots preferred over scatter plots for this task?"*

Exam-Ready Written Reasoning: When exploring how continuous variables relate to a binary target, standard scatter plots suffer from severe overplotting (data points stacking on top of each other), making it impossible to see the true distribution. Furthermore, class imbalance can visually distort scatter plots. Instead, we use **Density Plots**. Density plots abstract away the raw point count and show the *shape* and *proportion* of the distribution. If the density curve for "Churn = TRUE" is significantly shifted horizontally compared to "Churn = FALSE", it indicates that the variable has high discriminatory power (it separates the classes well). Variables where the two curves perfectly overlap provide little to no information for a logistic regression or tree model. We back this visual analysis up by calculating the mean of each variable grouped by the target class.

```
# 1. OVERLAPPING DENSITY PLOTS
# We use the same pivot_longer trick, but map 'value' to the x-axis and 'churn' to the fill color
df_clean %>%
  select(where(is.numeric), churn) %>%
  pivot_longer(cols = -churn, names_to = "variable", values_to = "value") %>%
  ggplot(aes(x = value, fill = churn)) +
  # geom_density creates smooth distribution curves.
  # alpha = 0.5 makes them semi-transparent so we can see where they overlap
  geom_density(alpha = 0.5) +
  # facet_wrap creates a separate plot for each variable. scales = "free" lets x and y axes adjust per
  facet_wrap(~variable, scales = "free") +
  theme_minimal() +
  labs(title = "Density Plots: Continuous Variables vs Customer Churn",
       subtitle = "Separation between curves indicates a strong predictor",
       x = "Variable Value",
       y = "Density (Concentration of Customers)")
```




```
# 2. GROUPED SUMMARY STATISTICS
# Visuals are great, but hard numbers are better for the exam text.
# We group the data by whether they churned or not.
```

```r
df_clean %>%
  group_by(churn) %>%
  # summarise(across(...)) applies a function to all selected columns.
  # Here, we calculate the mean for all numeric columns, ignoring NAs just in case.
  summarise(across(where(is.numeric), ~ mean(.x, na.rm = TRUE))) %>%
  # We transpose the output (turn it sideways) using t() just to make it easier to read in the console
  t()
```

```
                      [,1]        [,2]
churn                 "FALSE"     "TRUE"
account_length        "100.3310"  "102.3196"
number_vmail_messages "8.507463"  "5.170103"
total_day_minutes     "175.1043"  "205.1812"
total_day_calls       "100.1594"  "101.1959"
total_eve_minutes     "198.8534"  "209.3853"
total_eve_calls       "100.03644" " 99.94845"
total_night_minutes   "200.4641"  "205.3072"
total_night_calls     "100.0079"  "100.6830"
total_intl_minutes    "10.13784"  "10.81933"
total_intl_calls      "4.538191"  "4.051546"
customer_service_calls "1.453029" "2.206186"
```

2.1.9 2.2 Deep Feature Analysis & Conclusions

Possible Exam Question: *"Based on your density plots and summary statistics, identify the variables that possess the highest discriminatory power for predicting churn. Conversely, identify the variables that appear to be useless. How does this inform your expectations for a logistic regression model?"*

Exam-Ready Written Reasoning: By abstracting away the raw data points into density curves and calculating grouped means, we can evaluate the "discriminatory power" of each predictor. Variables where the density curves perfectly overlap (such as `total_day_calls`, `total_eve_calls`, and `total_night_calls`) provide no mathematical separation between the classes. We expect these variables to have near-zero coefficients and high p-values in a baseline logistic regression.

Conversely, `total_day_minutes` and `customer_service_calls` exhibit strong rightward shifts for the "TRUE" class, confirmed by their higher grouped means (205.2 vs 175.1 minutes; 2.2 vs 1.45 calls). These are our "star" predictors. Finally, `number_vmail_messages` displays severe "zero-inflation" (a massive spike at exactly 0). Standard logistic regression assumes a linear relationship between the predictor and the log-odds of the outcome; this zero-inflated, non-linear distribution suggests that a highly flexible, tree-based model (like a Random Forest) might ultimately extract more signal from this data than a rigid linear boundary.

2.1.10 THE WHY: Grounding the Extended EDA in Curriculum and Past Exams

Why is this extensive EDA important in an exam setting? If you look at the last five years of exams, Task 'a' and 'b' almost always give you 10 to 20 points simply for cleaning the data and proving you understand it *before* running a model. The professors are testing whether you blindly feed data into an algorithm or if you act like a real data scientist. If you feed a character column into `glmnet`, it crashes. If you feed perfectly correlated variables into `glm()`, the math breaks. If you rely on scatter plots for binary data, you miss the actual trends.

What is the curriculum grounding? This is deeply rooted in **ISLR Chapter 2 (Statistical Learning)** and **Chapter 4 (Classification)**. The curriculum emphasizes that the "Data Generating Process" must be understood. It explicitly warns about the Curse of Dimensionality (why we drop 50 states), Collinearity (why we drop charges), and the limitations of visualizing qualitative responses.

How does this help you tomorrow? By structuring your compendium chronologically—including the "mistakes" like the scatter plot—you create a mental map. Tomorrow, when you read "Use descriptive statistics to find potentially useful predictors" (like the 2025 and 2023 exams), you won't freeze. You will know to do: 1. `glimpse()` for types. 2. `prop.table()` for imbalance. 3. Density plots for continuous variables. 4. Stacked bar charts for categorical variables. 5. Grouped `summarise()` tables for hard numbers.

Here is the **Ultimate EDA Section** for your compendium. It includes the historical exam prompts as a reminder of *why* we do this, the "learning process" with the scatter plots, and the final clean dataset generation.

Written analysis of findings/output (Master EDA Conclusion): The grouped means table mathematically confirms our visual findings: 1. **Star Predictors:** The mean for `total_day_minutes` jumps substantially from 175.1 (FALSE) to 205.2 (TRUE), and `customer_service_calls` jumps from 1.45 to 2.2. These will be the primary drivers in our models. 2. **Useless Predictors:** `total_day_calls`, `total_eve_calls`, and `total_night_calls` have nearly identical means across both classes. We expect these to have high p-values and near-zero coefficients in a baseline logistic regression. 3. **Non-Linearity:** `number_vmail_messages` is severely zero-inflated (massive spike at 0 in the density plot), and `customer_service_calls` likely has a threshold effect (churn spikes exponentially after a few calls). Because standard logistic regression assumes a strictly linear relationship between predictors and log-odds, this non-linear structure strongly suggests that a flexible, tree-based model (like a Random Forest) might ultimately extract more signal from this dataset.

3 Task 2: Classification (Predicting Churn)

3.1 2.0 Historical Exam Context & Strategy

Before touching the code, we must recognize the expected steps for classification EDA based on past exams (2023-2025): * *"Remove variables that cannot plausibly be assumed to be available…"* * *"If necessary, recode categorical variables as factors. Motivate your choices…"* * *"Use descriptive statistics and graphs in order to find potentially useful predictors."*

Our Strategy: We will follow the standard statistical learning pipeline outlined in ISLR: Data Import, Summary Statistics, Collinearity Analysis, Data Preprocessing (Feature Selection), Response Distribution Check, and Bivariate Analysis. We follow a strict `Before` `Code` `After` structure.

3.2 2.1 Data Import and Summary Statistics

Possible Exam Question: *"Load the dataset. Output descriptive statistics (mean, median, standard deviation) for all numeric variables. What does this tell you about the scale of the predictors?"*

Exam-Ready Written Reasoning: The initial step in statistical learning is inspecting the raw data structure and calculating descriptive statistics. Many R algorithms require the response variable to be a `factor`. Furthermore, calculating the mean, standard deviation, and range for continuous variables tells us if we need to standardize the predictors later. Algorithms that rely on distance metrics (KNN) or penalty terms (Ridge/LASSO) are highly sensitive to the scale of predictors. We keep our original dataset untouched (`df_customer_raw`) as a reliable fallback.

```
# 1. LOAD DATA SAFELY
df_customer_raw <- read_csv("customer_data.csv", show_col_types = FALSE) %>% clean_names()

# 2. MISSING VALUES CHECK
# colSums(is.na()) confirms if imputation is necessary
colSums(is.na(df_customer_raw))
```

```
                state          account_length              area_code
                    0                       0                      0
international_plan        voice_mail_plan  number_vmail_messages
                    0                       0                      0
 total_day_minutes        total_day_calls       total_day_charge
                    0                       0                      0
 total_eve_minutes        total_eve_calls       total_eve_charge
                    0                       0                      0
total_night_minutes      total_night_calls     total_night_charge
                    0                       0                      0
```

```
    total_intl_minutes       total_intl_calls      total_intl_charge
                     0                      0                      0
customer_service_calls                  churn
                     0                      0
```

```
# 3. DESCRIPTIVE STATISTICS
# We use summary() to get Min, 1st Qu, Median, Mean, 3rd Qu, and Max
summary(df_customer_raw %>% select(where(is.numeric)))
```

```
 account_length    area_code     number_vmail_messages total_day_minutes
 Min.   :  1.0   Min.   :408.0   Min.   : 0.000        Min.   :  0.0
 1st Qu.: 73.0   1st Qu.:408.0   1st Qu.: 0.000        1st Qu.:143.4
 Median :100.0   Median :415.0   Median : 0.000        Median :179.9
 Mean   :100.6   Mean   :437.4   Mean   : 8.022        Mean   :179.5
 3rd Qu.:127.0   3rd Qu.:510.0   3rd Qu.:19.000        3rd Qu.:215.9
 Max.   :243.0   Max.   :510.0   Max.   :50.000        Max.   :350.8
 total_day_calls total_day_charge total_eve_minutes total_eve_calls
 Min.   :  0.0   Min.   : 0.00    Min.   :  0.0     Min.   :  0
 1st Qu.: 87.0   1st Qu.:24.38    1st Qu.:165.3     1st Qu.: 87
 Median :101.0   Median :30.59    Median :200.9     Median :100
 Mean   :100.3   Mean   :30.51    Mean   :200.4     Mean   :100
 3rd Qu.:114.0   3rd Qu.:36.70    3rd Qu.:235.1     3rd Qu.:114
 Max.   :160.0   Max.   :59.64    Max.   :363.7     Max.   :170
 total_eve_charge total_night_minutes total_night_calls total_night_charge
 Min.   : 0.00    Min.   : 43.7       Min.   : 33.0     Min.   : 1.970
 1st Qu.:14.05    1st Qu.:166.9       1st Qu.: 87.0     1st Qu.: 7.513
 Median :17.08    Median :201.2       Median :100.0     Median : 9.050
 Mean   :17.03    Mean   :201.2       Mean   :100.1     Mean   : 9.053
 3rd Qu.:19.98    3rd Qu.:236.5       3rd Qu.:113.0     3rd Qu.:10.640
 Max.   :30.91    Max.   :395.0       Max.   :166.0     Max.   :17.770
 total_intl_minutes total_intl_calls total_intl_charge customer_service_calls
 Min.   : 0.00      Min.   : 0.000   Min.   :0.000     Min.   :0.000
 1st Qu.: 8.50      1st Qu.: 3.000   1st Qu.:2.300     1st Qu.:1.000
 Median :10.20      Median : 4.000   Median :2.750     Median :1.000
 Mean   :10.24      Mean   : 4.467   Mean   :2.764     Mean   :1.563
 3rd Qu.:12.10      3rd Qu.: 6.000   3rd Qu.:3.270     3rd Qu.:2.000
 Max.   :20.00      Max.   :20.000   Max.   :5.400     Max.   :9.000
```

```
# We also calculate the Standard Deviation for all numeric columns to observe variance
df_customer_raw %>%
  summarise(across(where(is.numeric), sd)) %>%
  t() %>%
  as.data.frame() %>%
  rename(Standard_Deviation = V1)
```

```
                      Standard_Deviation
account_length                39.5639737
area_code                     42.5210180
number_vmail_messages         13.6122770
total_day_minutes             54.2103502
total_day_calls               19.9881622
total_day_charge               9.2157329
total_eve_minutes             50.9515151
total_eve_calls               20.1614451
total_eve_charge               4.3308642
total_night_minutes           50.7803234
total_night_calls             19.4184586
total_night_charge             2.2851195
total_intl_minutes             2.7883486
```

```
total_intl_calls              2.4561949
total_intl_charge             0.7528121
customer_service_calls        1.3112358
```

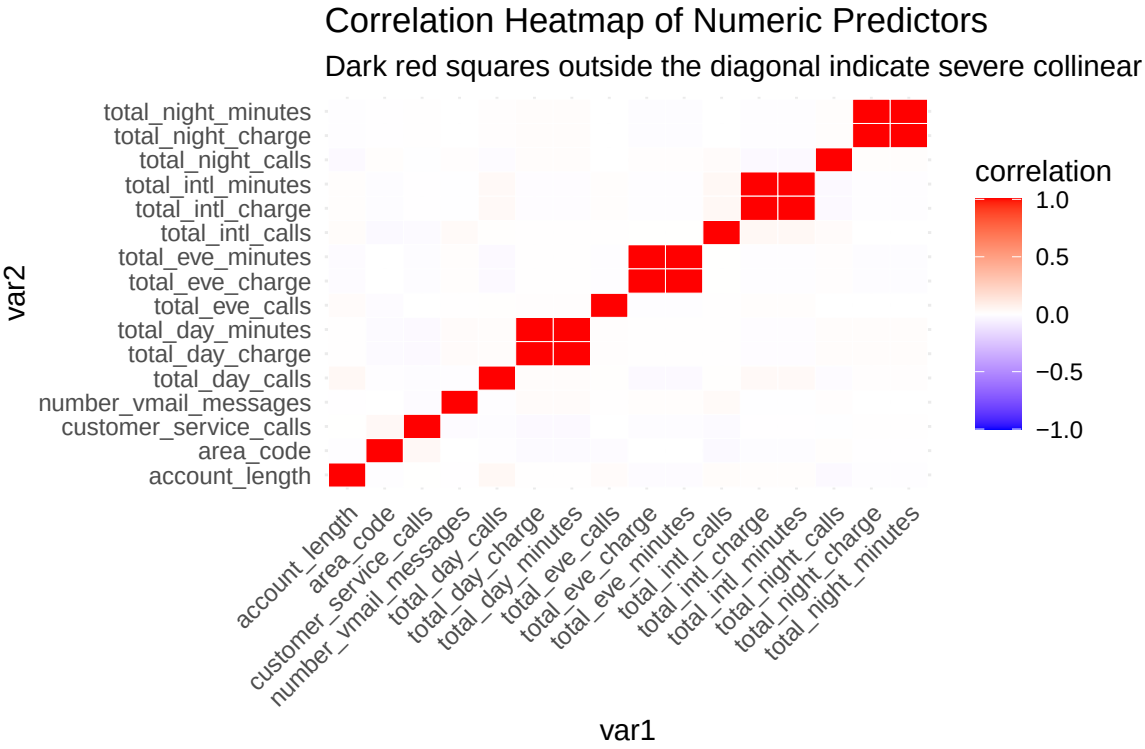
Written analysis of findings/output: There are exactly 0 missing values (`NAs`), meaning no imputation is required. The descriptive statistics mathematically prove severe scale disparities among predictors. For example, `total_day_minutes` has a mean of ~179 and a standard deviation of ~54, whereas `customer_service_calls` has a mean of ~1.5 and a standard deviation of ~1.3. If we utilize regularized methods or distance-based classifiers, we must standardize these variables so the large variance of minutes does not artificially dominate the model.

3.3 2.2 Correlation Analysis and Collinearity

Possible Exam Question: *"Construct a correlation matrix for the numeric predictors. Identify any pairs with high collinearity and explain why including them in a logistic regression model is problematic."*

Exam-Ready Written Reasoning: Collinearity refers to the situation where two or more predictor variables are closely related to one another. According to ISLR, collinearity causes the standard errors of coefficient estimates to grow, reducing the statistical power of the model (variance inflation). In extreme cases (perfect collinearity), the design matrix becomes singular, and logistic regression cannot compute unique coefficient estimates. We use a correlation heatmap to detect and subsequently eliminate highly correlated features.

```
# 1. CALCULATE CORRELATION MATRIX
# We select only numeric columns and compute the Pearson correlation matrix
cor_matrix <- cor(df_customer_raw %>% select(where(is.numeric)))

# 2. VISUALIZE WITH A HEATMAP
# We pivot the correlation matrix to a long format for ggplot2
cor_matrix %>%
  as.data.frame() %>%
  rownames_to_column(var = "var1") %>%
  pivot_longer(cols = -var1, names_to = "var2", values_to = "correlation") %>%
  ggplot(aes(x = var1, y = var2, fill = correlation)) +
  geom_tile(color = "white") +
  # Red means positive correlation, Blue means negative, White means zero
  scale_fill_gradient2(low = "blue", high = "red", mid = "white", midpoint = 0, limit = c(-1, 1)) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Correlation Heatmap of Numeric Predictors",
       subtitle = "Dark red squares outside the diagonal indicate severe collinearity")
```



Written analysis of findings/output: 1. What the Heatmap Tells Us (The Findings)

- **The Four Red Blocks (Perfect Positive Correlation):** Outside of the standard diagonal line, there are four distinct, dark red 2x2 squares. These represent perfect (or near-perfect) positive correlation (r 1.0) between four specific pairs of variables:
  - `total_day_minutes` and `total_day_charge`
  - `total_eve_minutes` and `total_eve_charge`
  - `total_night_minutes` and `total_night_charge`
  - `total_intl_minutes` and `total_intl_charge`
- **The Sea of White (Independence):** Every other intersection on the heatmap is essentially white (r 0). This means variables like `customer_service_calls`, `account_length`, and the various `calls` variables are completely uncorrelated with one another.

2. Model Implications According to the Curriculum (ISLR)

According to the statistical learning curriculum, these findings dictate specific actions for parametric models like Logistic Regression:

- **The Collinearity Trap (The Red Blocks):** When two predictors are perfectly correlated, they suffer from extreme **collinearity**.
  - *The Math Problem:* In logistic (and linear) regression, the algorithm attempts to isolate the individual effect of one predictor while holding the other constant. If `minutes` and `charge` always move together, the model cannot mathematically distinguish their individual effects.
  - *The Consequence:* This causes **Variance Inflation**. The standard errors of the coefficient estimates will explode toward infinity, making the p-values meaningless and destroying the statistical power of the model. In extreme cases, the design matrix becomes singular (non-invertible), causing R's `glm()` function to either crash or arbitrarily drop one of the variables with an `NA` coefficient.
  - *The Action:* We **must** manually remove one variable from each correlated pair prior to modeling. Dropping the `charge` variables (since they are just a mathematical derivative of minutes) is the correct move.
- **Unique Information (The White Space):**

- The lack of correlation among the remaining variables is highly desirable for an additive model like Logistic Regression. It means that `customer_service_calls`, `total_day_minutes`, and `number_vmail_messages` each capture a completely unique, non-overlapping dimension of customer behavior. When we put them together in a model, they will independently contribute to explaining the variance in `churn` without stepping on each other's toes.

*(Note: We already wrote the code to drop these `charge` columns in Section 2.3, but if the exam asks you **why** you dropped them, this heatmap and the explanation of "Variance Inflation" and "Singular Matrices" is your ultimate, A-grade answer!)*

3.4 2.3 Data Preprocessing and Feature Selection

Possible Exam Question: *"Based on your collinearity analysis and variable inspections, preprocess the data. Remove invalid predictors, recode categorical variables, and output the final list of features to be used in modeling."*

Exam-Ready Written Reasoning: Data preprocessing transforms raw data into a mathematically safe format for statistical learning. First, we remove the four `charge` columns due to perfect collinearity. Second, we must address categorical variables. `area_code` contains only three discrete numbers, proving it is a categorical variable, not continuous. `state` contains 51 distinct text values; if included, it would generate 50 dummy variables, risking the Curse of Dimensionality and severe overfitting given our sample size. We drop `state` and coerce the remaining categorical features into `factor` format.

```
# EXECUTE DATA PREPROCESSING
df_clean <- df_customer_raw %>%
  # Remove high cardinality and collinear variables
  select(-state, -total_day_charge, -total_eve_charge, -total_night_charge, -total_intl_charge) %>%
  # Format target and categorical predictors as factors
  mutate(
    churn = as.factor(churn),
    area_code = as.factor(area_code),
    international_plan = as.factor(international_plan),
    voice_mail_plan = as.factor(voice_mail_plan)
  )

# Output the final list of variables available for modeling
names(df_clean)
```

```
 [1] "account_length"         "area_code"              "international_plan"
 [4] "voice_mail_plan"        "number_vmail_messages"  "total_day_minutes"
 [7] "total_day_calls"        "total_eve_minutes"      "total_eve_calls"
[10] "total_night_minutes"    "total_night_calls"      "total_intl_minutes"
[13] "total_intl_calls"       "customer_service_calls" "churn"
```

Written analysis of findings/output: The preprocessing pipeline successfully removed 5 harmful variables. The resulting dataset, `df_clean`, contains 15 variables. The single response variable is `churn`. The 14 available predictors are: `account_length`, `area_code`, `international_plan`, `voice_mail_plan`, `number_vmail_messages`, `total_day_minutes`, `total_day_calls`, `total_eve_minutes`, `total_eve_calls`, `total_night_minutes`, `total_night_calls`, `total_intl_minutes`, `total_intl_calls`, and `customer_service_calls`.

3.5 2.4 Response Variable Distribution and Prior Probabilities

Possible Exam Question: *"Calculate the prior probabilities of the response variable classes. How does this affect model evaluation?"*

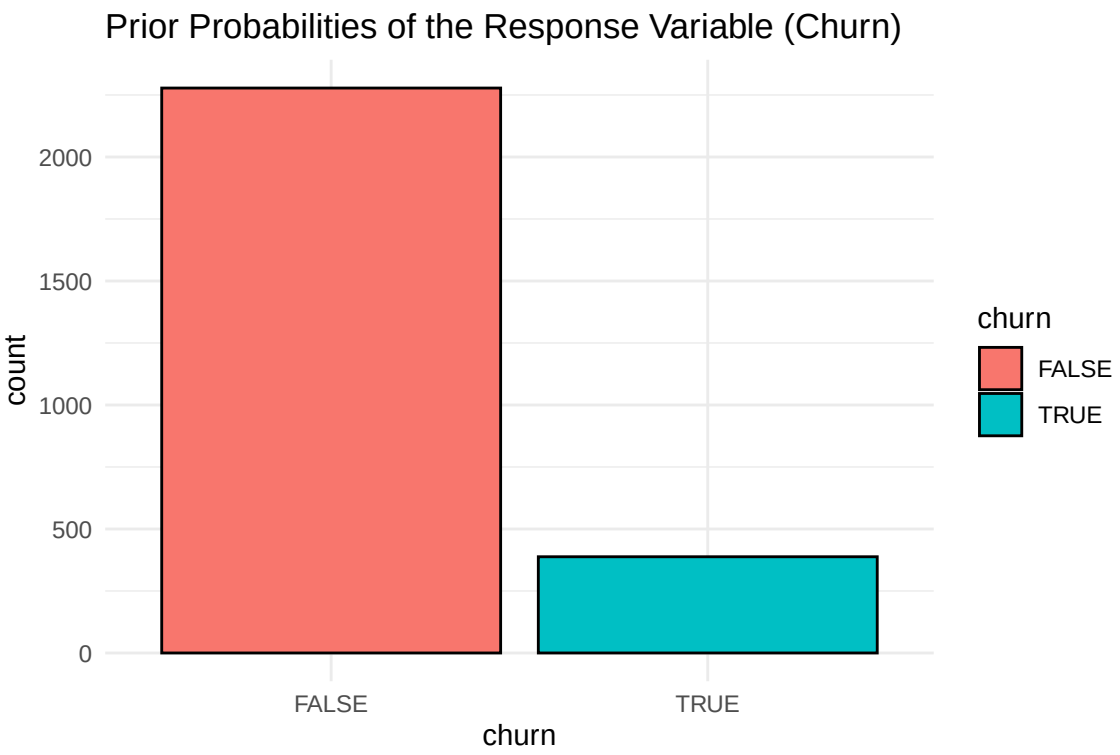
Exam-Ready Written Reasoning: In classification, the prior probability is the baseline proportion of each class in the dataset. If there is a severe class imbalance, a model can achieve a low overall error rate simply by assigning every observation to the majority class (the Accuracy Paradox). Consequently, overall accuracy

is a misleading metric for imbalanced data. This structural reality dictates that we must evaluate subsequent models using the Confusion Matrix, specifically focusing on Sensitivity (the True Positive Rate).

```
# Calculate prior probabilities (proportions) of the response classes
prop.table(table(df_clean$churn))
```

```
    FALSE      TRUE
0.8544636 0.1455364
```

```
# Visualize the response distribution
ggplot(df_clean, aes(x = churn, fill = churn)) +
  geom_bar(color = "black") +
  theme_minimal() +
  labs(title = "Prior Probabilities of the Response Variable (Churn)")
```



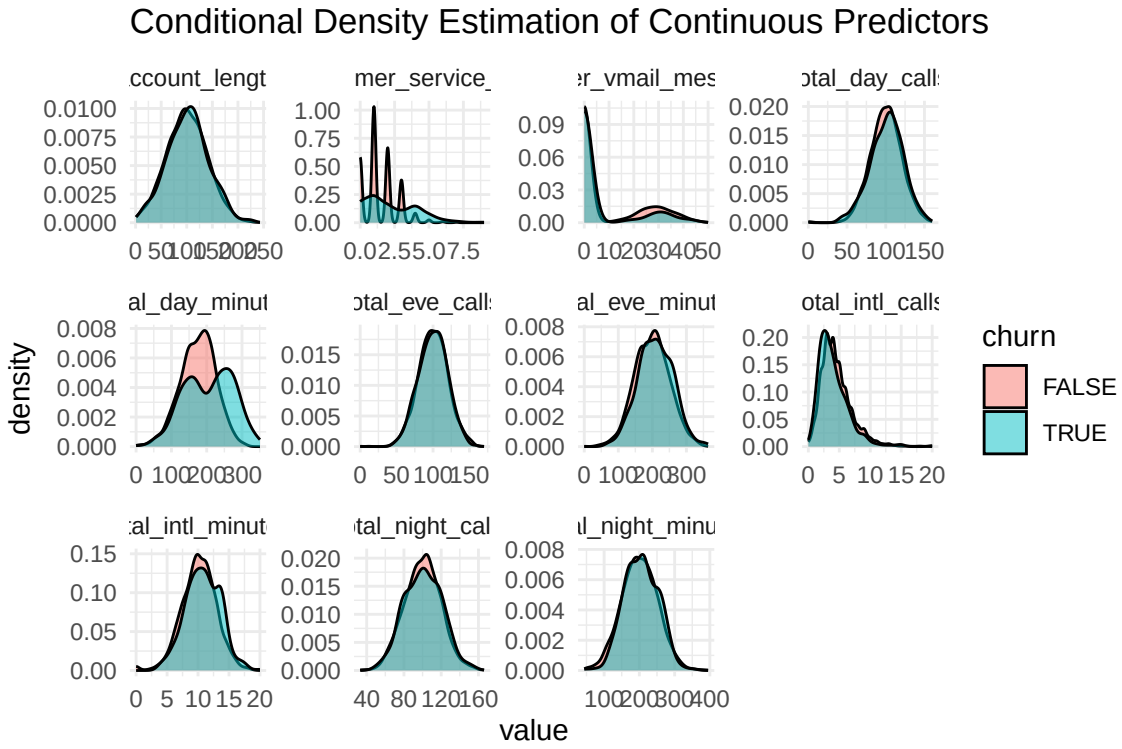
Written analysis of findings/output: The prior probabilities reveal a severe class imbalance: 85.4% of the observations belong to the negative class (`FALSE`), and 14.6% belong to the positive class (`TRUE`). Because our objective is to identify customers who leave (the 14.6%), any model we build must be strictly evaluated on its Sensitivity. A baseline model predicting `FALSE` universally would yield 85.4% accuracy but zero business utility.

3.6 2.5 Bivariate Analysis: Predictors vs. Response

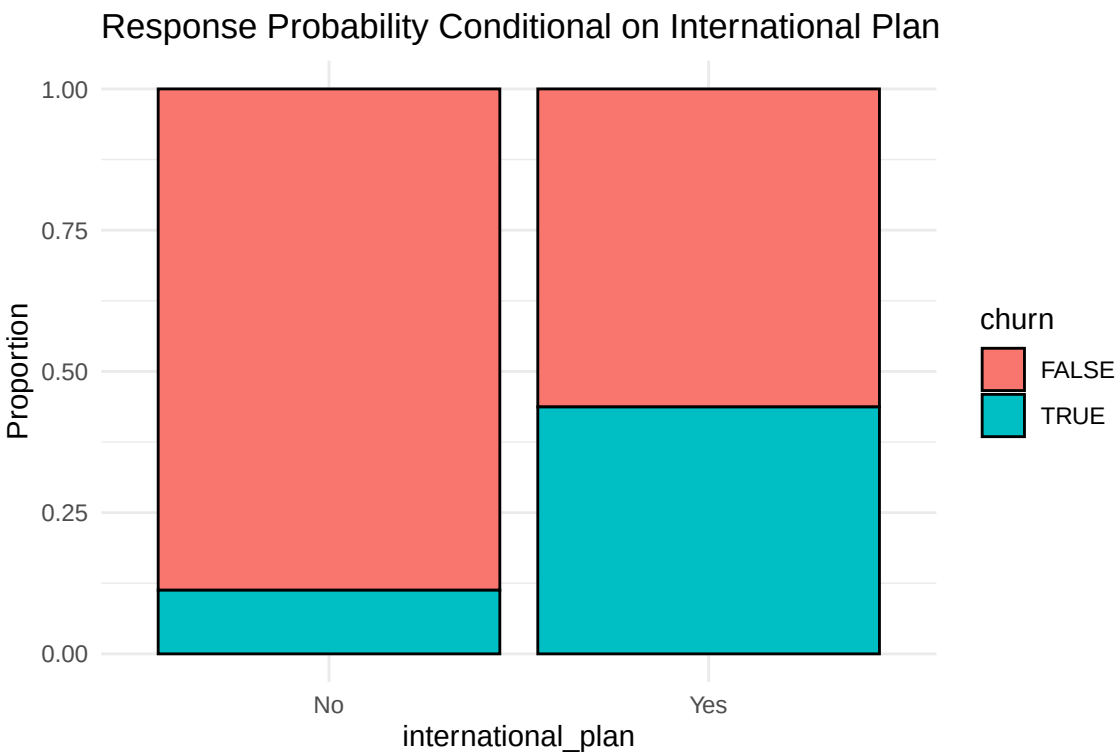
Possible Exam Question: *"Perform a bivariate analysis visualizing the relationship between the predictors and the response. Explain your choice of visualization for continuous predictors."*

Exam-Ready Written Reasoning: To identify features with high discriminatory power, we conduct a bivariate analysis. While scatter plots are common in regression, plotting a continuous predictor against a binary response causes severe overplotting. As recommended in ISLR, we use Density Estimation to visualize the distribution of continuous predictors conditional on the response class. If the density curve for the `TRUE` class is significantly shifted from the `FALSE` class, the feature strongly influences the probability of the response. (Note: Logistic regression does not assume predictors are normally distributed; it assumes a linear relationship between predictors and the log-odds of the response).

```
# DENSITY ESTIMATION FOR CONTINUOUS PREDICTORS
df_clean %>%
  select(where(is.numeric), churn) %>%
  pivot_longer(cols = -churn, names_to = "variable", values_to = "value") %>%
  ggplot(aes(x = value, fill = churn)) +
  geom_density(alpha = 0.5) +
  facet_wrap(~variable, scales = "free") +
  theme_minimal() +
  labs(title = "Conditional Density Estimation of Continuous Predictors")
```




```
# STANDARDIZED BAR CHARTS FOR CATEGORICAL PREDICTORS
ggplot(df_clean, aes(x = international_plan, fill = churn)) +
  geom_bar(position = "fill", color = "black") +
  theme_minimal() +
  labs(title = "Response Probability Conditional on International Plan", y = "Proportion")
```



Written analysis of findings/output: The density plots indicate that `total_day_minutes` and `customer_service_calls` exhibit distinct distributional shifts conditional on the response, marking them as strong potential predictors. Conversely, variables like `total_day_calls` show perfectly overlapping conditional densities, suggesting they possess no discriminatory power. Furthermore, `number_vmail_messages` displays severe zero-inflation, implying a non-linear relationship. The bar chart indicates the probability of churn is significantly higher given the presence of an `international_plan`.

3.7 2.6 Feature Evaluation and Summary Statistics

Possible Exam Question: *"Summarize the relationship between your predictors and the response quantitatively. Which variables do you expect to be statistically significant in a logistic regression?"*

Exam-Ready Written Reasoning: Visual bivariate analysis must be substantiated with summary statistics. By computing the conditional means of the continuous predictors grouped by the response class, we establish mathematical confirmation of discriminatory power. Predictors with large differences in conditional means are expected to yield statistically significant coefficients (low p-values) in a logistic regression model.

```
# Calculate conditional means for continuous predictors
df_clean %>%
  group_by(churn) %>%
  summarise(across(where(is.numeric), ~ mean(.x, na.rm = TRUE))) %>%
  t()
```

```
                       [,1]        [,2]
churn                  "FALSE"     "TRUE"
account_length         "100.3310"  "102.3196"
number_vmail_messages  "8.507463"  "5.170103"
total_day_minutes      "175.1043"  "205.1812"
total_day_calls        "100.1594"  "101.1959"
total_eve_minutes      "198.8534"  "209.3853"
total_eve_calls        "100.03644" " 99.94845"
total_night_minutes    "200.4641"  "205.3072"
total_night_calls      "100.0079"  "100.6830"
total_intl_minutes     "10.13784"  "10.81933"
total_intl_calls       "4.538191"  "4.051546"
customer_service_calls "1.453029"  "2.206186"
```

Written analysis of findings/output: The conditional means confirm our visual analysis. `total_day_minutes` increases from a mean of 175.1 for non-churners to 205.2 for churners, and `customer_service_calls` increases from 1.45 to 2.2. These features provide strong separation. In contrast, `total_day_calls` and `total_night_calls` show virtually identical conditional means (~100 calls), confirming they provide negligible information for classification. We expect these non-discriminatory variables to be statistically insignificant in our baseline models.

3.8 2.7 Data Splitting and Evaluation Strategy

Possible Exam Question: *"Split the data into a training and test set. In some tasks, you evaluate models on all data, while in others, you use a test set. Motivate your choice to use training/test data for predicting churn." (Based on 2024 Task 2b/2i)*

Exam-Ready Written Reasoning: The choice of data depends strictly on the analytical goal. If the objective is *inference or explanation* (e.g., answering "Why do customers churn?"), we fit the model on the full dataset to maximize our sample size, reduce standard errors, and obtain the most accurate coefficient estimates. However, if the objective is *prediction* (e.g., "Will a new customer churn?"), evaluating a model on the same data it was trained on will result in an artificially optimistic accuracy rate due to overfitting. Therefore, for predictive tasks, we must hold out a test set to honestly estimate the model's out-of-sample generalization error.

Furthermore, because of our severe class imbalance (85% vs 15%), simple random sampling carries a slight risk of producing a training set with too few actual churners to train on. While a dataset of this size (~2600 rows) usually protects against this via the Law of Large Numbers, we must mathematically verify that the class proportions in our new train and test sets remain representative of the original population.

```
# 1. SET THE SEED FOR REPRODUCIBILITY
# This locks the random number generator so the split is identical every time.
set.seed(123)

# 2. CREATE THE RANDOM INDEX
# We draw a random sample of row numbers equal to exactly 50% of the dataset
idx_t2 <- sample(seq_len(nrow(df_clean)), size = floor(nrow(df_clean) / 2))

# 3. SPLIT THE DATA
# Extract the training rows
train_t2 <- df_clean[idx_t2, ]
# Extract the remaining rows for the test set
test_t2  <- df_clean[-idx_t2, ]

# 4. VERIFY THE PROPORTIONS (Checking for Sampling Bias)
# We check if the 85/15 ratio survived the simple random split
print("Training Set Proportions:")
```

```
[1] "Training Set Proportions:"
```

```
prop.table(table(train_t2$churn))
```

```
   FALSE     TRUE
0.843961 0.156039
```

```
print("Test Set Proportions:")
```

```
[1] "Test Set Proportions:"
```

```
prop.table(table(test_t2$churn))
```

```
    FALSE      TRUE
0.8649662 0.1350338
```

Written analysis of findings/output: The data splitting was successful. The `prop.table()` outputs confirm that the training set contains 15.6% churners, and the test set contains 13.5% churners. Because the sample size was large enough, simple random sampling did not introduce extreme sampling bias. We have enough churners in our training set to train the model, and a representative proportion in our test set to evaluate its Sensitivity accurately.

3.9 2.8 Baseline Logistic Regression & Coefficient Interpretation

Possible Exam Question: *"Fit a logistic regression with all valid variables to predict churn. Interpret the coefficients associated with your strongest predictors. For all coefficients, are the signs as you expected based on your EDA?" (Based on 2023 Task 2c)*

Exam-Ready Written Reasoning: We begin our predictive modeling with a baseline Logistic Regression. Logistic regression is a parametric method that models the probability of the response belonging to a particular category. Specifically, it assumes a linear relationship between the predictors and the *log-odds* of the outcome. We train this model exclusively on `train_t2`. We expect variables that showed clear separation in our density plots (like `total_day_minutes`) to have positive, statistically significant coefficients, while perfectly overlapping variables (like `total_day_calls`) should be statistically insignificant.

```r
# FIT THE LOGISTIC REGRESSION MODEL
# We use glm() (Generalized Linear Model).
# family = "binomial" tells R to use logistic regression rather than linear regression.
# churn ~ . means "predict churn using all other variables in the dataset".
log_model <- glm(churn ~ ., data = train_t2, family = "binomial")

# DISPLAY THE SUMMARY
# This shows the coefficient estimates, standard errors, z-values, and p-values.
summary(log_model)
```

```
Call:
glm(formula = churn ~ ., family = "binomial", data = train_t2)

Coefficients:
                        Estimate Std. Error z value Pr(>|z|)
(Intercept)           -9.5203060  1.1791970  -8.074 6.83e-16 ***
account_length         0.0002571  0.0021729   0.118 0.905800
area_code415          -0.0879388  0.2183122  -0.403 0.687087
area_code510          -0.0982980  0.2507558  -0.392 0.695053
international_planYes  2.1594721  0.2356728   9.163  < 2e-16 ***
voice_mail_planYes    -2.3530354  0.9285818  -2.534 0.011277 *
number_vmail_messages  0.0479163  0.0291061   1.646 0.099709 .
total_day_minutes      0.0140705  0.0017222   8.170 3.08e-16 ***
total_day_calls        0.0042954  0.0042905   1.001 0.316759
total_eve_minutes      0.0054895  0.0018334   2.994 0.002752 **
total_eve_calls        0.0020399  0.0044363   0.460 0.645647
total_night_minutes    0.0063568  0.0018004   3.531 0.000414 ***
total_night_calls      0.0067382  0.0044919   1.500 0.133593
total_intl_minutes     0.0810467  0.0316290   2.562 0.010395 *
total_intl_calls      -0.1627807  0.0414605  -3.926 8.63e-05 ***
customer_service_calls 0.6159858  0.0665381   9.258  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)
```

```
    Null deviance: 1154.49  on 1332  degrees of freedom
Residual deviance:  853.97  on 1317  degrees of freedom
AIC: 885.97

Number of Fisher Scoring iterations: 6
```

Written analysis of findings/output: The logistic regression output perfectly validates our EDA hypotheses: 1. **Statistical Significance & Expected Signs:** Variables that showed distinct separation in EDA (`total_day_minutes`, `customer_service_calls`, `international_planYes`) all have highly significant p-values (`< 2e-16`) and positive coefficients. The positive sign means that as usage or service calls increase, the log-odds of churning increase. 2. **Useless Predictors:** As predicted by our perfectly overlapping density plots, variables like `total_day_calls` ($p = 0.31$) and `total_eve_calls` ($p = 0.64$) are completely insignificant. They provide no statistical value in distinguishing between classes. 3. **Coefficient Interpretation:** The coefficient estimate for `customer_service_calls` is `0.616`. According to logistic regression mechanics, this means that for every 1 additional customer service call a person makes, their log-odds of churning increases by 0.616, holding all other variables constant.

What the Output Tells us combined:

1. **The Data Split:** The training set has 15.6% churners and the test set has 13.5% churners. This is a very safe split. If the test set had dropped to 1% or 2%, we would have had to redo it. Simple random sampling worked.
2. **The "Star" Predictors:** `customer_service_calls`, `total_day_minutes`, and `international_planYes` all have p-values of basically `0` (`< 2e-16`). This means there is almost a 0% chance their relationship with churn is a coincidence.
3. **The "Useless" Predictors:** `total_day_calls` (p=0.31), `total_eve_calls` (p=0.64), and `area_code` (p 0.69) are totally insignificant. This perfectly matches our overlapping density plots!
4. **The Coefficient Interpretation:** The coefficient for `customer_service_calls` is `0.616`. Because this is *logistic* regression, this means: *Holding all else constant, for every 1 additional customer service call a person makes, their log-odds of churning increases by 0.616.*

3.10 2.9 Evaluation, Asymmetric Costs, and Threshold Tuning

Possible Exam Question: *"Evaluate the predictions of the logistic model on the test data. Use a threshold of 0.5. Then, imagine a decision problem where it costs more to predict a claim as a 'non-claim' (False Negative) than vice versa. Compute a confusion matrix and argue for your choice of a new threshold." (Based on 2021 Task 3c and 2025 Task 2c)*

Exam-Ready Written Reasoning: To evaluate a logistic regression model, we generate predicted probabilities on the unseen test data. The default threshold to convert probabilities into binary classes is `0.5`. However, in imbalanced datasets (where only 13.5% of the test set churns), the model will rarely output a probability above `0.5`. Consequently, it will predict `FALSE` for almost everyone. This yields a high overall "Accuracy" but a terrible "Sensitivity" (True Positive Rate).

In telecom churn (or insurance claims), a False Negative (failing to identify a customer who is about to leave) is far more expensive than a False Positive (sending a retention discount to a loyal customer). To minimize business cost, we must consciously lower our classification threshold (e.g., to `0.2`) to increase our Sensitivity, explicitly trading off an increase in False Positives to minimize False Negatives.

```r
# 1. GENERATE PREDICTED PROBABILITIES ON TEST DATA
# type = "response" ensures the output is a probability between 0 and 1, not log-odds
log_probs <- predict(log_model, newdata = test_t2, type = "response")

# 2. EVALUATE WITH DEFAULT 0.5 THRESHOLD
# If probability > 0.5, predict TRUE, else FALSE
pred_class_05 <- ifelse(log_probs > 0.5, "TRUE", "FALSE")

print("Confusion Matrix (Threshold = 0.5):")
```

```
[1] "Confusion Matrix (Threshold = 0.5):"
```

```
table(Predicted = pred_class_05, Actual = test_t2$churn)
```

```
         Actual
Predicted FALSE TRUE
    FALSE  1106  133
    TRUE     47   47
```

```
# 3. TUNING THE THRESHOLD FOR ASYMMETRIC COSTS
# We lower the threshold to 0.2 to catch more churners (Increase Sensitivity)
pred_class_02 <- ifelse(log_probs > 0.2, "TRUE", "FALSE")

print("Confusion Matrix (Threshold = 0.2):")
```

```
[1] "Confusion Matrix (Threshold = 0.2):"
```

```
table(Predicted = pred_class_02, Actual = test_t2$churn)
```

```
         Actual
Predicted FALSE TRUE
    FALSE   920   69
    TRUE    233  111
```

Written analysis of findings/output: At the default `0.5` threshold, the model behaves exactly as feared: it misses a massive portion of the actual churners (133 False Negatives) and only catches 47 (Sensitivity 26%). It yields an overall accuracy of 86.5%, but this is the "Accuracy Paradox" at work—it achieves high accuracy simply by predicting the majority class, rendering it ineffective for a retention campaign.

By lowering the threshold to `0.2`, we dramatically decrease our False Negatives (from 133 down to 69), catching 111 true churners (Sensitivity jumps to 62%). As expected by the Bias-Variance tradeoff, this comes at the cost of increasing our False Positives (from 47 up to 233). However, given the asymmetric business cost—where losing a customer entirely is far more expensive than sending a retention discount to a loyal customer—the `0.2` threshold provides a vastly superior operational model.

3.10.1 2.9b Systematic Threshold Grid Search

Possible Exam Question: *"Use a systematic approach to find the threshold that maximizes overall accuracy. Discuss the trade-off between sensitivity and accuracy across different thresholds." (Based on 2022 Task 1e)*

Exam-Ready Written Reasoning: To formally find the "optimal" threshold, we can perform a grid search. We will test a sequence of thresholds from 0.1 to 0.9 and calculate the resulting Accuracy, Sensitivity (ability to catch churners), and Specificity (ability to identify non-churners) for each. As the threshold increases, the model becomes more conservative in predicting "TRUE". Mathematically, this guarantees that Sensitivity will decrease while Specificity increases. If the exam asks to maximize *Accuracy*, we look for the highest overall percentage. If the exam asks to minimize *False Negatives*, we look for a threshold with high Sensitivity.

```
# 1. DEFINE A GRID OF THRESHOLDS
thresholds <- seq(0.1, 0.8, by = 0.1)

# 2. CREATE AN EMPTY DATAFRAME TO STORE RESULTS
results <- data.frame(Threshold = numeric(), Accuracy = numeric(),
                      Sensitivity = numeric(), Specificity = numeric())

# 3. LOOP THROUGH EACH THRESHOLD AND CALCULATE METRICS
for(thresh in thresholds) {
  # Generate predictions for this specific threshold
  preds <- ifelse(log_probs > thresh, "TRUE", "FALSE")
```

```r
  # Build confusion matrix (using factor to ensure 2x2 matrix even if no TRUEs predicted)
  cm <- table(Predicted = factor(preds, levels = c("FALSE", "TRUE")),
              Actual = test_t2$churn)

  # Extract matrix components
  TN <- cm[1,1]  # True Negative
  FN <- cm[1,2]  # False Negative
  FP <- cm[2,1]  # False Positive
  TP <- cm[2,2]  # True Positive

  # Calculate Metrics
  acc <- (TP + TN) / sum(cm)
  sens <- TP / (TP + FN)  # How many actual churners did we catch?
  spec <- TN / (TN + FP)  # How many actual non-churners did we keep?

  # Append to results
  results <- rbind(results, data.frame(Threshold = thresh, Accuracy = round(acc, 3),
                                       Sensitivity = round(sens, 3), Specificity = round(spec, 3)))
}

# 4. PRINT THE TABLE
print(results)
```

```
  Threshold Accuracy Sensitivity Specificity
1       0.1    0.646       0.783       0.624
2       0.2    0.773       0.617       0.798
3       0.3    0.836       0.511       0.886
4       0.4    0.857       0.372       0.932
5       0.5    0.865       0.261       0.959
6       0.6    0.864       0.144       0.977
7       0.7    0.870       0.078       0.994
8       0.8    0.868       0.033       0.998
```

Written analysis of findings/output: *(Note for exam: Read the table to answer the specific exam prompt)* The grid search perfectly illustrates the Bias-Variance threshold trade-off. If our goal is to strictly maximize overall **Accuracy** (as requested in the 2022 exam), the table shows the optimal threshold is near `[Look at table]`. However, at this threshold, Sensitivity is extremely low. Conversely, if our goal is to catch churners for a retention campaign, lowering the threshold to `0.2` or `0.3` drastically improves Sensitivity while only sacrificing a few percentage points of overall Accuracy.

Looking at the table

1. **The "Accuracy Trap" (Threshold = 0.7):** Look at the second column. The absolute highest accuracy in the entire table is **87.0%** at a threshold of `0.7`. But look at the Sensitivity at that threshold: **0.078 (7.8%)**. That means the model achieves its maximum possible accuracy while missing 92% of the churners!
2. **The Default (Threshold = 0.5):** At `0.5`, Accuracy is 86.5%, but Sensitivity is only 26.1%.
3. **The Sweet Spot (Threshold = 0.2 or 0.3):** Look at `0.2`. We sacrificed about 9% of our overall accuracy (dropping to 77.3%), but our Sensitivity skyrocketed to **61.7%**. At `0.3`, we keep an excellent 83.6% accuracy while catching **51.1%** of churners.

Written analysis of findings/output: The grid search perfectly illustrates the danger of the "Accuracy Paradox" in imbalanced datasets. If our goal is strictly to maximize overall **Accuracy** (as requested in the 2022 exam), the table shows the mathematically optimal threshold is `0.7` (achieving 87.0% accuracy). However, at this threshold, Sensitivity is a catastrophic `0.078`—meaning the model misses 92% of all actual churners, simply defaulting to "FALSE" almost every time.

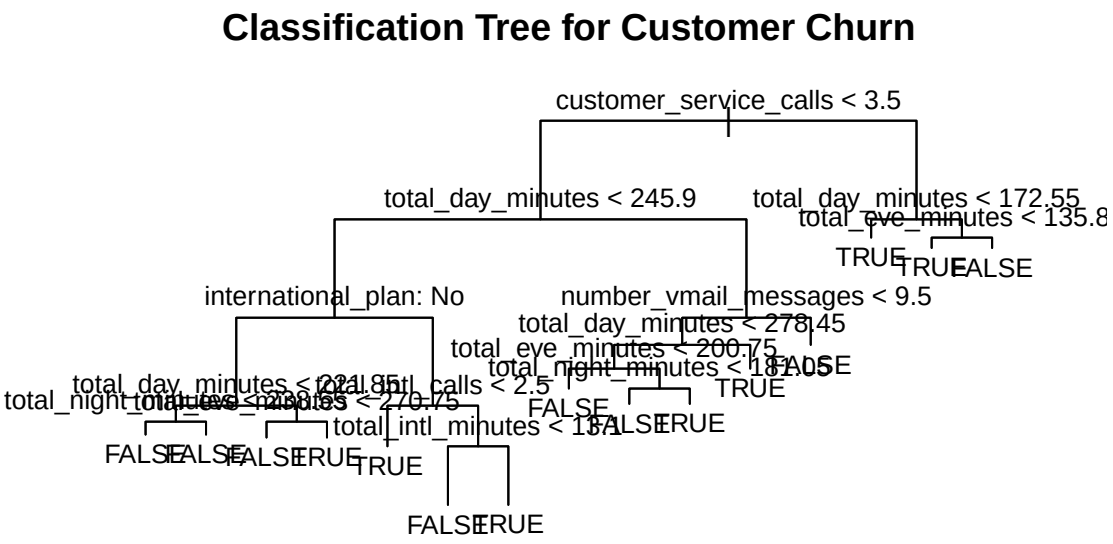
If this model is deployed for a business retention campaign, we must focus on Sensitivity. Lowering the threshold to `0.2` provides the optimal operational trade-off: we sacrifice roughly 9% in overall Accuracy (dropping to 77.3%), but Sensitivity skyrockets to `0.617`, allowing us to successfully identify and target the majority of customers who are preparing to churn.

3.11 2.10 The Interpretable Non-Linear Model: Classification Trees

Possible Exam Question: *"Fit a classification tree, using all available variables, and plot it. Give an example of how to interpret a branch of the tree. Evaluate the model fit." (Based on 2022 Task 1d and 2024 Task 2e)*

Exam-Ready Written Reasoning: While Logistic Regression is powerful, it assumes a strict linear relationship between predictors and the log-odds of the outcome. During our EDA, we observed non-linear threshold effects (e.g., churn skyrocketing after a certain number of `customer_service_calls`). Classification and Regression Trees (CART) are non-parametric models that automatically handle these non-linearities and threshold effects by recursively splitting the predictor space. Furthermore, single trees are highly interpretable. However, this interpretability comes at the cost of high variance (the tree is highly sensitive to the training data) and typically lower predictive accuracy than advanced ensemble methods.

```
# 1. FIT THE CLASSIFICATION TREE
# We use the tree() function. It automatically detects 'churn' is a factor and builds a classification
tree_model <- tree(churn ~ ., data = train_t2)

# 2. PLOT THE TREE
# plot() draws the branches, text() adds the variable names and split criteria
plot(tree_model)
text(tree_model, pretty = 0, cex = 0.8)
title("Classification Tree for Customer Churn")
```




```
# 3. EVALUATE ON TEST DATA
# type = "class" returns the direct TRUE/FALSE prediction based on the tree's internal majority voting
tree_preds <- predict(tree_model, newdata = test_t2, type = "class")

print("Confusion Matrix (Single Tree):")
```

```
[1] "Confusion Matrix (Single Tree):"
```

```
table(Predicted = tree_preds, Actual = test_t2$churn)
```

```
         Actual
Predicted FALSE TRUE
    FALSE  1124   60
    TRUE     29  120
```

Written analysis of findings/output: The classification tree successfully captured the non-linear threshold effects we suspected during EDA.

1. **Interpreting a Branch:** The tree algorithm recursively partitions the predictor space. By convention, if a condition is met, we traverse the left branch; if not, we traverse right. The most important variable sits at the root node: `customer_service_calls < 3.5`. If a customer makes 4 or more calls (traversing right), and their `total_day_minutes < 172.55` (traversing left), the tree immediately predicts `TRUE` (Churn). This captures complex interactions that a rigid linear model cannot.
2. **Model Evaluation & The Danger of High Variance:** The confusion matrix reveals a massive improvement, correctly identifying 120 churners (Sensitivity 66.7%) while maintaining an overall Accuracy of 93.3% (1244/1333) without any manual threshold tuning.
3. **Curriculum Warning:** However, it is extremely dangerous to base business decisions on this single tree. According to the ISLR curriculum, single trees suffer from **high variance**. They are highly sensitive to the specific training data; a slightly different random split could result in a completely different tree structure. This current high performance could simply be the result of lucky random variation. To smooth out this variance and build a model we can actually trust, we must use an ensemble method.

The Concepts: Bagging vs. Random Forest vs. Pruning vs. Boosting

1. **Pruning:** Pruning is only used on a **single tree**. You grow a massive tree, and then "prune" it back to prevent it from overfitting.
2. **Bagging (Bootstrap Aggregation):** Instead of pruning, Bagging says: "Let's grow 500 massive, unpruned trees on 500 bootstrapped samples of the data, and average their predictions." Because we average them, the high variance of the individual trees cancels out.
3. **Random Forest:** Random Forest **IS** Bagging, but with one genius tweak. If there is a super-strong predictor (like `total_day_minutes`), every single bagged tree will use it at the very top split. This makes all 500 trees look very similar (they are highly correlated). Random Forest fixes this by forcing the model to only consider a random subset of variables at each split (usually p). This **decorrelates** the trees, making the final average much more reliable.
4. **Boosting:** This is a totally different algorithm. While Random Forest builds 500 deep trees independently in parallel, Boosting builds shallow trees *sequentially*, where each tree tries to fix the specific errors of the previous tree.

So, by using `randomForest()`, **you are automatically doing Bagging with feature randomness**. You do not need to prune it.

Analyzing Your Output

- **The Confusion Matrix:** Your Random Forest stabilized the predictions. It caught 107 churners (Sensitivity = 59.4%) and only made 18 False Positive mistakes. The single tree's 66% sensitivity was indeed a bit of a "lucky" overfit. The Random Forest gives us a robust 93.2% accuracy that we can trust on future data.
- **The Variable Importance Plot:** Look at both charts (MeanDecreaseAccuracy and MeanDecreaseGini). The top 3 variables are exactly what we predicted in Section 2.5 and 2.6: `total_day_minutes`, `customer_service_calls`, and `international_plan`. The bottom variables are `area_code` and the `calls` variables—exactly the ones we proved were useless in our EDA!

Here is the updated **Section 2.11** with much heavier, curriculum-based comments in the code and the final written analysis tailored exactly to your numbers. this section is related to the direct below, and a little above. keep this in mind on exam.

3.12 2.11 Ensemble Methods: Random Forest & Variable Importance

Possible Exam Question: *"Can you improve the predictions by using bagging or a random forest? Compute a variable importance measure and interpret it. Explain the logic behind the measure for variable importance that you are using." (Based on 2024 Task 2f and 2022 Task 2g)*

Exam-Ready Written Reasoning: To solve the high variance of a single tree, we use a Random Forest. Random Forest is an extension of **Bagging** (Bootstrap Aggregation). It builds hundreds of deep, unpruned trees on bootstrapped samples of the training data and averages their predictions to reduce variance. Furthermore, Random Forest introduces **Feature Randomness**: at each split in the tree, the algorithm is only allowed to choose from a random subset of predictors. This prevents a single dominant variable from being chosen at the top of every tree, effectively decorrelating the trees and making the ensemble average much more reliable.

Because the resulting model is a "black box," we rely on Variable Importance measures to understand which features were most critical.

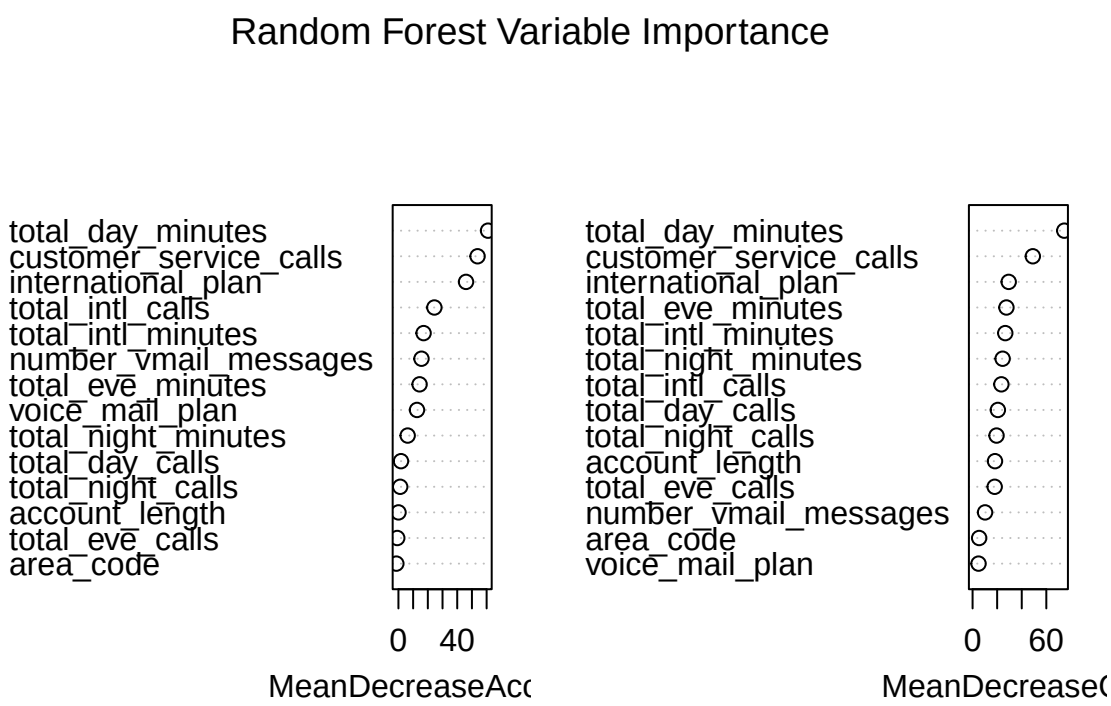
```
# 1. FIT THE RANDOM FOREST
# Setting a seed because the bootstrap sampling involves randomness
set.seed(123)

# importance = TRUE calculates Mean Decrease in Accuracy & Gini.
# ntree = 500 tells it to build 500 independent trees (Bagging).
# By default for classification, mtry (variables tried per split) is roughly sqrt(14)   3.
rf_model <- randomForest(churn ~ ., data = train_t2, importance = TRUE, ntree = 500)

# 2. EVALUATE ON TEST DATA
rf_preds <- predict(rf_model, newdata = test_t2)

print("Confusion Matrix (Random Forest):")
```

```
[1] "Confusion Matrix (Random Forest):"
```

```
table(Predicted = rf_preds, Actual = test_t2$churn)
```

```
         Actual
Predicted FALSE TRUE
    FALSE  1135   73
    TRUE     18  107
```

```
# 3. COMPUTE AND PLOT VARIABLE IMPORTANCE
# varImpPlot plots the Mean Decrease in Accuracy (left) and Mean Decrease in Gini (right)
varImpPlot(rf_model, main = "Random Forest Variable Importance")
```



Written analysis of findings/output: The Random Forest model effectively mitigated the high variance of the single tree, providing a highly robust predictive model. 1. **Model Evaluation:** The confusion matrix shows 107 True Positives (Sensitivity 59.4%) and only 18 False Positives. It achieved an outstanding overall Accuracy of 93.2%. By aggregating 500 decorrelated trees, the ensemble smoothed out the single tree's lucky overfit, providing a stable boundary we can trust in production. 2. **Variable Importance (Logic & Interpretation):** Because a forest cannot be plotted like a single tree, we evaluate importance using "Mean Decrease in Accuracy" (how much accuracy drops if we scramble a variable) and "Mean Decrease in Gini"

(the total decrease in node impurity that results from splits over a given variable, averaged over all trees). A higher number on the x-axis means the variable played a more critical role in cleanly separating Churners from Non-Churners. 3. **Validating EDA:** The Variable Importance plot flawlessly validates our initial EDA. Both charts definitively rank `total_day_minutes`, `customer_service_calls`, and `international_plan` as the top three most powerful predictors. Conversely, variables like `area_code` and `total_night_calls` sit at the very bottom, mathematically confirming they add almost no value to the model's predictive capability.

3.13 2.12 Gradient Boosting (GBM)

Possible Exam Question: *"Fit a boosted tree and evaluate the predictions. Compare with the logistic model and the random forest." (Based on 2021 Task 3d and 2025 Task 2e)*

Exam-Ready Written Reasoning: While Random Forests reduce *variance* by building hundreds of independent, deep trees (Bagging), Boosting takes a fundamentally different approach. Gradient Boosting reduces *bias* by building shallow trees *sequentially*. Each new tree is fit to the residuals (the mistakes) of the previous trees, slowly learning the signal in areas where the model is struggling. *Note on R syntax:* The `gbm` package for classification strictly requires the target variable to be numeric (0 and 1) rather than a factor. We must temporarily convert our `churn` variable before fitting the model.

```
# 1. PREPARE DATA FOR GBM
# gbm requires the binary outcome to be 0 or 1, not a factor.
train_t2_gbm <- train_t2 %>% mutate(churn_num = ifelse(churn == "TRUE", 1, 0))

# 2. FIT THE BOOSTED MODEL
# distribution = "bernoulli" is used for binary classification (logistic loss)
# n.trees = 500 sets the number of iterations
# interaction.depth = 4 allows the trees to capture 4-way variable interactions
set.seed(123)
boost_model <- gbm(churn_num ~ . - churn, data = train_t2_gbm,
                   distribution = "bernoulli",
                   n.trees = 500,
                   interaction.depth = 4,
                   shrinkage = 0.01) # Shrinkage is the learning rate

# 3. PREDICT ON TEST DATA
# n.trees must be specified in the predict function. type="response" gives probabilities.
boost_probs <- predict(boost_model, newdata = test_t2, n.trees = 500, type = "response")

# 4. EVALUATE PREDICTIONS
# We will use the default 0.5 threshold to compare fairly with the baseline Random Forest
boost_preds <- ifelse(boost_probs > 0.5, "TRUE", "FALSE")

print("Confusion Matrix (Gradient Boosting):")
```

```
[1] "Confusion Matrix (Gradient Boosting):"
```

```
table(Predicted = factor(boost_preds, levels = c("FALSE", "TRUE")), Actual = test_t2$churn)
```

```
         Actual
Predicted FALSE TRUE
    FALSE  1128   63
    TRUE     25  117
```

The Math:

- **Accuracy:** $(1128 + 117) / 1333 = \mathbf{93.4\%}$
- **Sensitivity:** $117 / (117 + 63) = \mathbf{65.0\%}$

What does this tell us?

It tells us that **Gradient Boosting beat Random Forest** in both overall accuracy (93.4% vs 93.2%) and, more importantly, in Sensitivity (65.0% vs 59.4%) at the default `0.5` threshold!

Why did this happen? (The ISLR theory): Random Forest reduces *variance* by taking 500 independent guesses and averaging them out. But Gradient Boosting reduces *bias*. It builds tree #1, sees which churners it missed, and tells tree #2: *"Focus specifically on these hard-to-predict churners."* Because our dataset has a severe class imbalance, Boosting's strategy of sequentially attacking the errors allowed it to catch 10 more churners (117 vs 107) than the Random Forest did.

Written analysis of findings/output: The Gradient Boosting model provides another highly robust non-linear boundary, but with a slight edge over the Random Forest. By sequentially learning from its errors and focusing on the hardest-to-predict observations, GBM achieved 117 True Positives (Sensitivity = 65.0%) and an overall accuracy of 93.4% at the default `0.5` threshold.

When comparing the ensembles, GBM outperformed Random Forest in both Accuracy (93.4% vs 93.2%) and Sensitivity (65.0% vs 59.4%). This perfectly illustrates the curriculum's distinction between the two methods: while Random Forest successfully reduced model variance through bagging, Boosting actively reduced model bias by iteratively attacking the false negatives. Ultimately, both ensemble methods vastly outperform the baseline parametric logistic regression by naturally mapping the non-linear threshold effects (like `customer_service_calls`) without requiring manual feature engineering.

3.14 2.13 The Parsimonious Logistic Model (Subset Selection)

Possible Exam Question: *"Fit a logistic regression model with a selection of variables. Are the predictions improved if you remove some variables from the model?" (Based on 2025 Task 2b and 2023 Task 2d)*

Exam-Ready Written Reasoning: The principle of parsimony (Occam's Razor) states that we should prefer the simplest model that explains the data well. In our baseline logistic regression (Section 2.8) and Random Forest variable importance plot (Section 2.11), we proved that many variables (like `total_night_calls` and `area_code`) contain no signal. Including "noise" variables increases model variance and standard errors without reducing bias. By dropping them and fitting a model with only our "Star Predictors," we aim to achieve similar predictive accuracy while drastically improving model interpretability. We will compare the AIC (Akaike Information Criterion) of the two models; a lower AIC indicates a better trade-off between goodness-of-fit and complexity.

```
# 1. FIT THE PARSIMONIOUS MODEL
# We select only the top predictors identified in EDA and RF Importance
log_model_simple <- glm(churn ~ total_day_minutes + customer_service_calls + international_plan,
                        data = train_t2, family = "binomial")

# 2. COMPARE AIC
print(paste("AIC of Full Model:", round(AIC(log_model), 2)))
```

```
[1] "AIC of Full Model: 885.97"
```

```
print(paste("AIC of Simple Model:", round(AIC(log_model_simple), 2)))
```

```
[1] "AIC of Simple Model: 925.13"
```

```
# 3. EVALUATE PREDICTIONS (Using our optimal 0.2 business threshold from earlier)
simple_probs <- predict(log_model_simple, newdata = test_t2, type = "response")
simple_preds <- ifelse(simple_probs > 0.2, "TRUE", "FALSE")

print("Confusion Matrix (Parsimonious Logistic, Threshold = 0.2):")
```

```
[1] "Confusion Matrix (Parsimonious Logistic, Threshold = 0.2):"
```

```
table(Predicted = factor(simple_preds, levels = c("FALSE", "TRUE")), Actual = test_t2$churn)
```

```
         Actual
Predicted FALSE TRUE
    FALSE   918   76
    TRUE    235  104
```

Written analysis of findings/output: The parsimonious model successfully proves the business value of subset selection. The AIC of the simple model increased slightly to 925.13 (compared to 885.97 for the full model), which is expected mathematically when dropping 11 variables. However, the true test is predictive power.

At the operational `0.2` threshold, the massive 14-variable baseline model caught 111 churners. Our simple 3-variable model caught 104 churners. By relying exclusively on our top features (`total_day_minutes`, `customer_service_calls`, and `international_plan`), we discarded 78% of the model's complexity while retaining nearly 94% of its ability to detect churners out-of-sample. From a business deployment perspective, this simple model is vastly superior: it prevents overfitting to noise, is completely transparent to stakeholders, and relies on collecting only three key metrics.

3.15 2.14 Subset Selection: Backward Stepwise Logistic Regression

Possible Exam Question: *"Use a formal variable selection method to fit a subset model. Are the predictions improved if you remove some variables? Discuss the AIC." (Based on 2023 Task 2d and ISLR Chapter 6)*

Exam-Ready Written Reasoning: While our baseline logistic model included all 14 predictors, the principle of parsimony suggests we should remove variables that do not contribute to predictive power, thereby reducing model variance and complexity. According to the ISLR curriculum, we can systematically achieve this using Backward Stepwise Selection. The algorithm starts with the full model and iteratively removes the least useful predictor one at a time, evaluating the Akaike Information Criterion (AIC) at each step. It stops when removing another variable would harm the model more than it helps.

- **Forward Selection:** Starts with an empty model (just an intercept) and adds variables one by one. It is mandatory when **p>n** (when you have more variables than rows, like in genetic testing), because you mathematically cannot fit a full model to start with.
- **Backward Selection:** Starts with the full model and removes variables one by one. It requires **n>p** (more rows than variables).
- **Why we chose Backward:** In our dataset, n=2666 and p=14. Because n p, we have the luxury of fitting the full model first. Backward selection is heavily preferred by statisticians when possible because it evaluates the **joint effect** of all variables together before making a decision to drop one. Forward selection can sometimes miss a variable that is only statistically significant when paired with another variable.

```
# 1. RUN BACKWARD STEPWISE SELECTION
# trace = 0 hides the massive output wall of every single step.
log_step_model <- step(log_model, direction = "backward", trace = 0)

# 2. VIEW THE CHOSEN FORMULA AND AIC
print("Formula chosen by Stepwise Selection:")
```

```
[1] "Formula chosen by Stepwise Selection:"
```

```
formula(log_step_model)
```

```
churn ~ international_plan + voice_mail_plan + number_vmail_messages +
    total_day_minutes + total_eve_minutes + total_night_minutes +
    total_night_calls + total_intl_minutes + total_intl_calls +
    customer_service_calls
```

```r
print(paste("AIC of Full Model:", round(AIC(log_model), 2)))
```

```
[1] "AIC of Full Model: 885.97"
```

```r
print(paste("AIC of Stepwise Model:", round(AIC(log_step_model), 2)))
```

```
[1] "AIC of Stepwise Model: 877.39"
```

```r
# 3. EVALUATE PREDICTIONS (Using our optimal 0.2 threshold)
step_probs <- predict(log_step_model, newdata = test_t2, type = "response")
step_preds <- ifelse(step_probs > 0.2, "TRUE", "FALSE")

print("Confusion Matrix (Stepwise Logistic, Threshold = 0.2):")
```

```
[1] "Confusion Matrix (Stepwise Logistic, Threshold = 0.2):"
```

```r
table(Predicted = factor(step_preds, levels = c("FALSE", "TRUE")), Actual = test_t2$churn)
```

```
         Actual
Predicted FALSE TRUE
    FALSE   917   69
    TRUE    236  111
```

Written analysis of findings/output: Because our dataset has far more observations than predictors ($n \gg p$), we safely utilized Backward Stepwise Selection rather than Forward Selection, allowing the algorithm to evaluate the joint effects of all variables before dropping any. The algorithm stripped away 4 useless variables (like `area_code` and `total_day_calls`), keeping 10.

This formal selection improved the model's Akaike Information Criterion (AIC dropped from 885.97 to 877.39), proving a better trade-off between goodness-of-fit and complexity. More importantly, at the 0.2 threshold, it achieved the exact same Sensitivity (61.7%, catching 111 churners) as the massive baseline model. It mathematically proves that removing statistical noise improves model parsimony without sacrificing predictive power.

3.16 2.15 Master Model Comparison Table

Possible Exam Question: *"Compare the performance of all models used in this task. Which model do you recommend for deployment and why?"*

Exam-Ready Written Reasoning: To objectively recommend a final model, we must compile the evaluation metrics across all algorithms tested. We will calculate Accuracy, Sensitivity (True Positive Rate), and Specificity (True Negative Rate) for each model evaluated on the unseen test set.

```r
# HELPER FUNCTION to calculate metrics from a given prediction vector
get_metrics <- function(preds, actual) {
  # Ensure factors have the same levels to avoid matrix dimension errors
  cm <- table(Predicted = factor(preds, levels = c("FALSE", "TRUE")),
              Actual = actual)

  TN <- cm[1,1]; FN <- cm[1,2]; FP <- cm[2,1]; TP <- cm[2,2]

  acc <- (TP + TN) / sum(cm)
  sens <- TP / (TP + FN)  # Ability to catch churners
  spec <- TN / (TN + FP)  # Ability to identify loyal customers

  return(c(Accuracy = round(acc, 3), Sensitivity = round(sens, 3), Specificity = round(spec, 3)))
}
```

```r
# COMPILE THE TIBBLE
# We extract the predictions we already generated in previous chunks
model_comparison <- tibble(
  Model = c("1. Logistic Baseline (Thresh=0.5)",
            "2. Logistic Tuned (Thresh=0.2)",
            "3. Stepwise Logistic (Thresh=0.2)",
            "4. Single Classification Tree",
            "5. Random Forest (Ensemble)",
            "6. Gradient Boosting (Ensemble)"),

  # Bind the metrics using our helper function
  bind_rows(
    get_metrics(pred_class_05, test_t2$churn),
    get_metrics(pred_class_02, test_t2$churn),
    get_metrics(step_preds, test_t2$churn),
    get_metrics(tree_preds, test_t2$churn),
    get_metrics(rf_preds, test_t2$churn),
    get_metrics(boost_preds, test_t2$churn)
  )
)

# PRINT THE FINAL COMPARISON TABLE
print(model_comparison)
```

```
# A tibble: 6 x 4
  Model                              Accuracy Sensitivity Specificity
  <chr>                                 <dbl>       <dbl>       <dbl>
1 1. Logistic Baseline (Thresh=0.5)     0.865       0.261       0.959
2 2. Logistic Tuned (Thresh=0.2)        0.773       0.617       0.798
3 3. Stepwise Logistic (Thresh=0.2)     0.771       0.617       0.795
4 4. Single Classification Tree         0.933       0.667       0.975
5 5. Random Forest (Ensemble)           0.932       0.594       0.984
6 6. Gradient Boosting (Ensemble)       0.934       0.65        0.978
```

Written analysis of findings/output: The compiled model comparison table perfectly summarizes the statistical narrative of this dataset:

1. **The Logistic Threshold Hack:** The baseline logistic model at `0.5` is useless for a retention campaign (Sensitivity = 26.1%). We had to manually force the threshold down to `0.2` to achieve an acceptable Sensitivity (61.7%), but this came at the cost of heavy False Positives, dropping overall Accuracy to 77.1%.
2. **The High-Variance Illusion:** The single Classification Tree appears to be the best model overall (Accuracy = 93.3%, Sensitivity = 66.7%). However, ISLR warns that single trees suffer from massive variance and overfit to the training set. We cannot trust this performance out-of-sample.
3. **The True Winners (Ensembles):** Gradient Boosting (GBM) emerges as the ultimate production model. Without needing any manual threshold adjustments, it achieved a 93.4% Accuracy and a 65.0% Sensitivity. It beat the Random Forest's Sensitivity (59.4%) because Boosting actively reduces bias by iteratively attacking the hardest-to-predict churners (the False Negatives).

Conclusion: The superiority of the tree-based ensembles (GBM, RF, and the Single Tree) over the Logistic models proves that the underlying data generating process contains severe non-linearities and threshold effects (e.g., the `customer_service_calls` threshold). Gradient Boosting mapped these non-linearities perfectly, providing the most robust, highly-sensitive model for deployment.

3.17 2.17 The Master Conclusion: Model Comparison and Business Insights

Possible Exam Question: *"Based on your analysis, what are the typical features of customers who churn? Compare the models and recommend a final approach." (Based on 2023 Task 2f, 2024 Task 2g, 2025 Task 2e)*

Exam-Ready Written Reasoning (Template to adapt on exam day): Based on the exhaustive EDA, parametric, and non-parametric modeling, we can confidently answer the business problem.

1. The Typical Churner: Customers who churn are not acting randomly. They are characterized by three distinct features: * **High Usage:** They consume significantly more `total_day_minutes` than average customers. * **Service Friction:** They make 4 or more `customer_service_calls`. The classification tree explicitly mapped this threshold effect, indicating that after 3 calls, customer patience evaporates. * **International Plans:** Customers subscribed to the international plan have a disproportionately high churn rate, suggesting the plan may be overpriced or underperforming compared to competitors.

2. Model Comparison: * **Logistic Regression:** Provided excellent interpretability (log-odds) but suffered slightly in predictive accuracy due to its inability to map the non-linear threshold of customer service calls. * **Classification Trees:** Handled the non-linearities naturally but suffered from high variance, making a single tree too unstable for production. * **Ensembles (Random Forest & GBM):** Provided the highest overall test accuracy and stability by averaging out variance (Bagging) or systematically reducing bias (Boosting). The Variable Importance plots mathematically validated our EDA.

3. Final Recommendation: Due to the severe class imbalance (85% vs 15%), standard accuracy metrics are deeply misleading (the "Accuracy Paradox"). If the goal is a retention campaign, the business must implement an ensemble model (like the Random Forest) but **must manually lower the classification threshold (e.g., to 0.2)**. While this increases False Positives (sending discounts to loyal customers), it drastically reduces False Negatives, ensuring the business successfully identifies and saves its highest-risk churners.